

백엔드 실무 학습

원리를 이해하고, 실패를 재현하고, 증거로 검증하는 21챕터 · 32일 학습
시스템

이 문서는 웹 학습 페이지와 동일한 구조화 JSON에서 생성됩니다. 읽기만 하지 말고 각 DAY의 실습·실패
주입·검증·회상 평가를 직접 수행하세요.

32일 로드맵

챕터	주제	DAY
01	학습 환경과 문제 해결 루틴	D01
02	복잡도·자료구조·Java 객체 모델	D02 · D03
03	OS·JVM·Java 동시성	D04 · D05
04	네트워크와 Web Infra	D06 · D07
05	Spring Container와 Web Runtime	D08 · D09
06	Spring Transaction과 DB 동시성	D10 · D11
07	SQL·실행계획·Oracle Partition	D12 · D13
08	Spring Batch와 대외 연계	D14 · D15
09	Array·String·Hash·Two Pointer	D16
10	Stack·Queue·Heap·Sort	D17
11	Binary Search·Greedy·Prefix	D18
12	Graph·Union-Find	D19
13	DP·Backtracking	D20
14	SQL 심화·Java 구현 품질·통합 코딩	D21 · D22
15	분산 실패 처리	D23
16	Kafka Architecture와 Reliability	D24 · D25
17	결제·정산·대사·원장	D26
18	System Design	D27
19	JPA·Redis·Container·Kubernetes·Security	D28
20	AI/RAG를 사용하는 백엔드	D29
21	실무 사례 분석·통합 평가·회상	D30 · D31 · D32

CHAPTER 01 · 학습 환경과 문제 해결 루틴

D01 · 학습 환경과 백엔드 시스템 관찰법

학습 환경과 백엔드 시스템 관찰법 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 JDK의 정의와 내부 동작을 설명합니다.
- 학습 환경과 백엔드 시스템 관찰법에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: JDK, PATH, stdout/stderr.
- 학습 환경과 백엔드 시스템 관찰법의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · JDK · PATH · stdout/stderr · exit code · baseline · log · metric · trace

Learn · 정의에서 관찰까지

정의	같은 실험을 다시 실행할 수 있도록 JDK/OS/환경변수/포트/입력/명령/예상출력을 고정한다.
왜 필요한가	학습 환경과 백엔드 시스템 관찰법 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	같은 실험을 다시 실행할 수 있도록 JDK/OS/환경변수/포트/입력/명령/예상출력을 고정한다. 로그는 사건의 문맥, 메트릭은 시간에 따른 수치, 트레이스는 여러 컴포넌트의 요청 경로를 보여준다.
선택 기준	컨테이너로 모든 환경을 즉시 숨기기보다 초반에는 실제 환경 차이를 눈으로 확인하는 편이 원인 분석 훈련에 유리하다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- JDK와 PATH의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 컨테이너로 모든 환경을 즉시 숨기기보다 초반에는 실제 환경 차이를 눈으로 확인하는 편이 원인 분석 훈련에 유리하다.

실패 조건

- 환경 차이를 확인하지 않고 코드를 먼저 수정하면 재현되지 않는 문제를 만들기 쉽다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D01 WORKED EXAMPLE

문제에서 검증까지

문제	두 PC에서 같은 Java 프로그램의 출력이 다르다. JDK, timezone, locale, 파일 인코딩부터 비교한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그-metric-state 정상-경계-실패-재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 환경 차이를 확인하지 않고 코드를 먼저 수정하면 재현되지 않는 문제를 만들기 쉽다.
조치	환경 정보를 README에 고정하고 동일 명령을 새 terminal에서 다시 실행해 결과를 검증한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

두 PC에서 같은 Java 프로그램의 출력이 다르다. JDK, timezone, locale, 파일 인코딩부터 비교한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 실습 디렉터리 생성
- java/javac/git/curl/docker 버전 기록

독립 단계

- 정상 Java 프로그램 실행
- 잘못된 경로로 실패 재현
- README만으로 재실행

명령

```
java -version
```

```
javac -version
```

```
git --version
```

```
curl --version
```


D01 VERIFY

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 환경 차이를 확인하지 않고 코드를 먼저 수정하면 재현되지 않는 문제를 만들기 쉽다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. JDK의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 같은 실험을 다시 실행할 수 있도록 JDK/OS/환경변수/포트/입력/명령/예상출력을 고정한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. 학습 환경과 백엔드 시스템 관찰법에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 컨테이너로 모든 환경을 즉시 숨기기보다 초반에는 실제 환경 차이를 눈으로 확인하는 편이 원인 분석 훈련에 유리하다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 환경 차이를 확인하지 않고 코드를 먼저 수정하면 재현되지 않는 문제를 만들기 쉽다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. 학습 환경과 백엔드 시스템 관찰법을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 두 PC에서 같은 Java 프로그램의 출력이 다르다. JDK, timezone, locale, 파일 인코딩부터 비교한다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 두 PC에서 같은 Java 프로그램의 출력이 다르다. JDK, timezone, locale, 파일 인코딩부터 비교한다.을 재현한 뒤 환경 정보를 README에 고정하고 동일 명령을 새 terminal에서 다시 실행해 결과를 검증한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

[Docker Documentation](#) · official-primary

D02 · 복잡도와 자료구조

복잡도와 자료구조 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 Big-O의 정의와 내부 동작을 설명합니다.
- 복잡도와 자료구조에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: Big-O, amortized, ArrayList.
- 복잡도와 자료구조의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · Big-O · amortized · ArrayList · LinkedList · HashMap · collision · Deque · LRU

Learn · 정의에서 관찰까지

정의	Big-O는 초 단위 속도가 아니라 입력 크기가 커질 때 연산량의 증가 형태를 설명한다.
왜 필요한가	복잡도와 자료구조 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Big-O는 초 단위 속도가 아니라 입력 크기가 커질 때 연산량의 증가 형태를 설명한다. HashMap은 hashCode로 bucket 후보를 좁힌 뒤 equals로 동등성을 확인하고, ArrayList는 연속 배열 기반이라 조회/순회가 유리하다.
선택 기준	O(1)이라도 상수 비용과 메모리 접근이 비쌀 수 있고, LinkedList는 삽입 이점보다 탐색/객체 오버헤드가 더 클 수 있다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- Big-O와 amortized의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- O(1)이라도 상수 비용과 메모리 접근이 비쌀 수 있고, LinkedList는 삽입 이점보다 탐색/객체 오버헤드가 더 클 수 있다.

실패 조건

- mutable key를 HashMap에 넣은 뒤 equality에 영향을 주는 값을 바꾸면 기존 bucket에서 찾지 못할 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D02 WORKED EXAMPLE

문제에서 검증까지

문제	요청 ID 빈도 집계기 List 선형 검색 반복으로 느리다. 키 조회 요구를 HashMap 누적으로 바꾼다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: mutable key를 HashMap에 넣은 뒤 equality에 영향을 주는 값을 바꾸면 기존 bucket에서 찾지 못할 수 있다.
조치	brute force와 HashMap 결과를 같은 입력으로 비교하고 N 증가에 따른 시간을 참고 수준으로 측정한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

요청 ID 빈도 집계기 List 선형 검색 반복으로 느리다. 키 조회 요구를 HashMap 누적으로 바꾼다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 빈도 집계 2문제
- mutable key 조회 실패 재현

독립 단계

- 잘못된 equals/hashCode 재현
- LinkedHashMap LRU 구현
- ArrayList/LinkedList 순회 비교

명령

```
javac src/CollectionLab.java
```

```
java -Xms256m -Xmx256m -cp src CollectionLab
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- mutable key를 HashMap에 넣은 뒤 equality에 영향을 주는 값을 바꾸면 기존 bucket에서 찾지 못할 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. Big-O의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Big-O는 초 단위 속도가 아니라 입력 크기가 커질 때 연산량의 증가 형태를 설명한다. 정의 뒤에 실제 목적과 경계를 붙여 설 명합니다.

2. 복잡도와 자료구조에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · O(1)이라도 상수 비용과 메모리 접근이 비쌀 수 있고, LinkedList는 삽입 이점보다 탐색/객체 오버헤드가 더 클 수 있다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · mutable key를 HashMap에 넣은 뒤 equality에 영향을 주는 값을 바꾸면 기존 bucket에서 찾지 못할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. 복잡도와 자료구조을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 복잡도와 자료구조의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 요청 ID 빈도 집계기 List 선형 검색 반복으로 느리다. 키 조회 요구를 HashMap 누적으로 바꾼다.을 재현한 뒤 brute force와 HashMap 결과를 같은 입력으로 비교하고 N 증가에 따른 시간을 참고 수준으로 측정한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

[Oracle Java SE 21 Documentation](#) · official-primary

D03 · Java 객체 모델과 API 계약

Java 객체 모델과 API 계약 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 identity의 정의와 내부 동작을 설명합니다.
- Java 객체 모델과 API 계약에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: identity, equality, immutable.
- Java 객체 모델과 API 계약의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · identity · equality · immutable · generic · PECS · checked/unchecked exception · Stream

Learn · 정의에서 관찰까지

정의	identity와 equality를 구분하고 equals를 재정의할 때 hashCode도 같은 동등성 기준을 사용한다.
왜 필요한가	Java 객체 모델과 API 계약 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	identity와 equality를 구분하고 equals를 재정의할 때 hashCode도 같은 동등성 기준을 사용한다. 불변 객체는 공유와 캐시 key에 유리하고 제네릭은 컴파일 시 타입 안전성을 높인다.
선택 기준	불변성은 복사 비용을 만들 수 있고 복잡한 wildcard는 API 가독성을 낮출 수 있다. Stream과 loop는 선언성, 할당, 예외 처리, 디버깅을 기준으로 선택한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- identity와 equality의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 불변성은 복사 비용을 만들 수 있고 복잡한 wildcard는 API 가독성을 낮출 수 있다. Stream과 loop는 선언성, 할당, 예외 처리, 디버깅을 기준으로 선택한다.

실패 조건

- 내부 예외를 그대로 외부에 노출하거나 같은 stack trace를 여러 계층에서 중복 기록하면 운영 로그의 신호 대 잡음 비율이 떨어진다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D03 WORKED EXAMPLE

문제에서 검증까지

문제	Money 값 객체의 amount/currency 비교 규칙이 여러 호출부에 중복되어 결과가 달라진다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 내부 예외를 그대로 외부에 노출하거나 같은 stack trace를 여러 계층에서 중복 기록하면 운영 로그의 신호 대 잡음 비율이 떨어진다.
조치	불변 값 객체에 생성 검증과 equality를 모으고 테스트로 계약을 고정한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

Money 값 객체의 amount/currency 비교 규칙이 여러 호출부에 중복되어 결과가 달라진다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 불변 Money 구현
- equals/hashCode 계약 테스트

독립 단계

- extends/super 예제
- 예외 변환 계층 구성
- Stream/loop 비교

명령

```
javac src/JavaContractLab.java
```

```
java -cp src JavaContractLab
```


실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 내부 예외를 그대로 외부에 노출하거나 같은 stack trace를 여러 계층에서 중복 기록하면 운영 로그의 신호 대 잡음 비율이 떨어진다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. identity의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · identity와 equality를 구분하고 equals를 재정의할 때 hashCode도 같은 동등성 기준을 사용한다. 정의 뒤에 실제 목적 과 경계를 붙여 설명합니다.

2. Java 객체 모델과 API 계약에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 불변성은 복사 비용을 만들 수 있고 복잡한 wildcard는 API 가독성을 낮출 수 있다. Stream과 loop는 선언성, 할당, 예외 처리, 디버깅을 기준으로 선택한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 내부 예외를 그대로 외부에 노출하거나 같은 stack trace를 여러 계층에서 중복 기록하면 운영 로그의 신호 대 잡음 비율이 떨어진다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Java 객체 모델과 API 계약을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Java 객체 모델과 API 계약의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · Money 값 객체의 amount/currency 비교 규칙이 여러 호출부에 중복되어 결과가 달라진다.을 재현한 뒤 불변 값 객체에 생성 검증과 equality를 모으고 테스트로 계약을 고정한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

D04 · OS·Process·Thread·JVM

OS·Process·Thread·JVM 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 process의 정의와 내부 동작을 설명합니다.
- OS·Process·Thread·JVM에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: process, thread, context switch.
- OS·Process·Thread·JVM의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · process · thread · context switch · virtual memory · stack · heap · metaspace · class loading · GC · STW

Learn · 정의에서 관찰까지

정의	프로세스는 독립 주소공간을 갖고 같은 프로세스의 스레드는 heap과 여러 자원을 공유하면서 각자 stack과 실행 상태를 가진다.
왜 필요한가	OS·Process·Thread·JVM 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	프로세스는 독립 주소공간을 갖고 같은 프로세스의 스레드는 heap과 여러 자원을 공유하면서 각자 stack과 실행 상태를 가진다. JVM은 class loading과 heap/stack/metaspase를 관리하고 GC는 도달 불가능 객체를 회수한다.
선택 기준	스레드를 늘리면 context switch와 공유 자원 경쟁이 늘 수 있다. GC pause만 줄이는 것보다 allocation rate 와 live set을 먼저 본다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- process와 thread의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 스레드를 늘리면 context switch와 공유 자원 경쟁이 늘 수 있다. GC pause만 줄이는 것보다 allocation rate 와 live set을 먼저 본다.

실패 조건

- CPU가 낮아도 외부 I/O, DB connection 또는 lock 대기로 요청이 쌓일 수 있다.

관찰

- thread dump·JFR·pool state
- queue·active count·rejection
- CPU·blocking I/O 구분

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D04 WORKED EXAMPLE

문제에서 검증까지

문제	응답 지연 중 CPU는 낮고 Thread dump에서 worker들이 connection 획득을 기다린다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. thread dump·JFR·pool state queue·active count·rejection CPU·blocking I/O 구분
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: CPU가 낮아도 외부 I/O, DB connection 또는 lock 대기로 요청이 쌓일 수 있다.
조치	worker 수만 늘리지 않고 thread state, connection pool, 외부 timeout의 균형을 조정한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

응답 지연 중 CPU는 낮고 Thread dump에서 worker들이 connection 획득을 기다린다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- RUNNABLE/WAITING/BLOCKED 재현
- jcmd Thread.print

독립 단계

- 작은 heap OOM 재현
- GC 로그 관찰
- OS RSS와 JVM heap 비교

명령

```
jps -l
```

```
jcmd <pid> Thread.print
```

```
jcmd <pid> GC.heap_info
```

```
java -Xmx64m -Xlog:gc* -cp src JvmLab
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- CPU가 낮아도 외부 I/O, DB connection 또는 lock 대기로 요청이 쌓일 수 있다.

관찰 증거

- thread dump·JFR·pool state
- queue·active count·rejection
- CPU·blocking I/O 구분

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. process의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 프로세스는 독립 주소공간을 갖고 같은 프로세스의 스레드는 heap과 여러 자원을 공유하면서 각자 stack과 실행 상태를 가 진다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. OS·Process·Thread·JVM 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 스레드를 늘리면 context switch와 공유 자원 경쟁이 늘 수 있다. GC pause만 줄이는 것보다 allocation rate와 live set 을 먼저 본다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · CPU가 낮아도 외부 I/O, DB connection 또는 lock 대기로 요청이 쌓일 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. OS·Process·Thread·JVM을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서 로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 응답 지연 중 CPU는 낮고 Thread dump에서 worker들이 connection 획득을 기다린다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 응답 지연 중 CPU는 낮고 Thread dump에서 worker들이 connection 획득을 기다린다.을 재현한 뒤 worker 수만 늘리지 않고 thread state, connection pool, 외부 timeout의 균형을 조정한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Official or Direct Reference](#) · official-primary

[Oracle Java SE 21 Documentation](#) · official-primary

D05 · Java 동시성과 실행자

Java 동시성과 실행자 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 JMM의 정의와 내부 동작을 설명합니다.
- Java 동시성과 실행자에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: JMM, happens-before, volatile.
- Java 동시성과 실행자의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · JMM · happens-before · volatile · synchronized · Atomic · CAS · ThreadPoolExecutor · queue · rejection · Virtual Thread

Learn · 정의에서 관찰까지

정의	JMM은 visibility/atomicity/ordering 규칙을 정의한다.
왜 필요한가	Java 동시성과 실행자 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	JMM은 visibility/atomicity/ordering 규칙을 정의한다. volatile은 visibility와 일부 ordering을 돕지만 count++ 전체의 atomicity를 보장하지 않고, synchronized는 상호 배제와 happens-before를 제공한다. ThreadPoolExecutor는 core→queue→max→rejection을 함께 이해해야 한다.
선택 기준	worker 수가 downstream capacity보다 많으면 처리량보다 대기만 늘 수 있다. Virtual Thread는 blocking I/O 동시성에 유리하지만 CPU 작업과 DB connection 제한을 없애지 않는다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- JMM와 happens-before의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- worker 수가 downstream capacity보다 많으면 처리량보다 대기만 늘 수 있다. Virtual Thread는 blocking I/O 동시성에 유리하지만 CPU 작업과 DB connection 제한을 없애지 않는다.

실패 조건

- unbounded queue는 maximumPoolSize 확장을 사실상 사용하지 못하게 만들 수 있고 대기 task가 메모 리를 압박할 수 있다.

관찰

- thread dump·JFR·pool state
- queue·active count·rejection
- CPU·blocking I/O 구분

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D05 WORKED EXAMPLE

문제에서 검증까지

문제	worker 100개인데 DB pool 20개라 connection wait만 증가한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. thread dump·JFR·pool state queue·active count·rejection CPU·blocking I/O 구분
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: unbounded queue는 maximumPoolSize 확장을 사실상 사용하지 못하게 만들 수 있고 대기 task가 메모 리를 압박할 수 있다.
조치	worker/queue/pool을 전체 capacity로 맞추고 timeout/backpressure를 함께 둔다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

worker 100개인데 DB pool 20개라 connection wait만 증가한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- volatile flag 비교
- Atomic vs synchronized increment

독립 단계

- 작은 queue rejection 재현
- queue 크기 변화 관찰
- Virtual Thread I/O 모사

명령

```
java -cp src ConcurrencyLab
```

```
jcmt <pid> Thread.print
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- unbounded queue는 maximumPoolSize 확장을 사실상 사용하지 못하게 만들 수 있고 대기 task가 메모 리를 압박할 수 있다.

관찰 증거

- thread dump·JFR·pool state
- queue·active count·rejection
- CPU·blocking I/O 구분

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. JMM의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · JMM은 visibility/atomicity/ordering 규칙을 정의한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Java 동시성과 실행자에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · worker 수가 downstream capacity보다 많으면 처리량보다 대기만 늘 수 있다. Virtual Thread는 blocking I/O 동시성 에 유리하지만 CPU 작업과 DB connection 제한을 없애지 않는다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · unbounded queue는 maximumPoolSize 확장을 사실상 사용하지 못하게 만들 수 있고 대기 task가 메모리를 압박할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Java 동시성과 실행자(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명 하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Java 동시성과 실행자의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · worker 100개인데 DB pool 20개라 connection wait만 증가한다.을 재현한 뒤 worker/queue/pool을 전체 capacity로 맞추고 timeout/backpressure를 함께 둔다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

[Oracle Java SE 21 Documentation](#) · official-primary

D06 · DNS·TCP·TLS·HTTP

DNS·TCP·TLS·HTTP 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 DNS의 정의와 내부 동작을 설명합니다.
- DNS·TCP·TLS·HTTP에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: DNS, TCP handshake, TLS.
- DNS·TCP·TLS·HTTP의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · DNS · TCP handshake · TLS · keep-alive · connect/read timeout · method · status · idempotency

Learn · 정의에서 관찰까지

정의	URL 호출은 DNS→TCP→TLS→HTTP 순서의 여러 경계를 지나며 각 단계의 실패 증상이 다르다.
왜 필요한가	DNS·TCP·TLS·HTTP 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	URL 호출은 DNS→TCP→TLS→HTTP 순서의 여러 경계를 지나며 각 단계의 실패 증상이 다르다. TCP는 byte stream이고 HTTP method 의미와 멍등성은 retry 설계와 연결된다.
선택 기준	keep-alive는 handshake 비용을 줄이지만 stale connection 관리가 필요하다. timeout은 DNS/connect/acquire/read 등 계층별로 budget을 나눈다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- DNS와 TCP handshake의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- keep-alive는 handshake 비용을 줄이지만 stale connection 관리가 필요하다. timeout은 DNS/connect/acquire/read 등 계층별로 budget을 나눈다.

실패 조건

- “timeout” 한 단어만 로그에 남기면 연결 실패인지 응답 지연인지 구분하기 어렵다.

관찰

- DNS·connect·TLS·HTTP 단계별 시간
- status와 upstream log
- timeout budget

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.

- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	외부 API가 30초 후 실패하지만 connect/read 구분이 없다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. DNS·connect·TLS·HTTP 단계별 시간 status와 upstream log timeout budget
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: "timeout" 한 단어만 로그에 남기면 연결 실패인지 응답 지연인지 구분하기 어렵다.
조치	curl -v와 단계별 timeout을 사용하고 맥당한 경우에만 bounded retry를 설계한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

외부 API가 30초 후 실패하지만 connect/read 구분이 없다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- curl -v 분석
- connection refused 재현

독립 단계

- read timeout 재현
- POST 중복과 idempotency key
- HTTP status 계약표

명령

```
curl -v https://example.com/
```

```
curl --connect-timeout 2 --max-time 5 http://127.0.0.1:65530/
```

```
nslookup example.com
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- “timeout” 한 단어만 로그에 남기면 연결 실패인지 응답 지연인지 구분하기 어렵다.

관찰 증거

- DNS-connect-TLS-HTTP 단계별 시간
- status와 upstream log
- timeout budget

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. DNS의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · URL 호출은 DNS→TCP→TLS→HTTP 순서의 여러 경계를 지나며 각 단계의 실패 증상이 다르다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. DNS-TCP-TLS-HTTP 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · keep-alive는 handshake 비용을 줄이지만 stale connection 관리가 필요하다. timeout은 DNS/connect/acquire/read 등 계층별로 budget을 나눈다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · “timeout” 한 단어만 로그에 남기면 연결 실패인지 응답 지연인지 구분하기 어렵다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. DNS-TCP-TLS-HTTP을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설 명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 외부 API가 30초 후 실패하지만 connect/read 구분이 없다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 외부 API가 30초 후 실패하지만 connect/read 구분이 없다.을 재현한 뒤 curl -v와 단계별 timeout을 사용하고 먹등한 경우에만 bounded retry를 설계한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Official or Direct Reference](#) · official-primary

[Official or Direct Reference](#) · official-primary

D07 · Nginx·Reverse Proxy·Load Balancer·Timeout

Nginx·Reverse Proxy·Load Balancer·Timeout 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 reverse proxy의 정의와 내부 동작을 설명합니다.
- Nginx·Reverse Proxy·Load Balancer·Timeout에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: reverse proxy, upstream, proxy_pass.
- Nginx·Reverse Proxy·Load Balancer·Timeout의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · reverse proxy · upstream · proxy_pass · load balancing · X-Forwarded-For · 502 · 504 · access/error log

Learn · 정의에서 관찰까지

정의	Reverse proxy는 TLS 종료, 라우팅, 정적 파일, header 전달, load balancing을 담당할 수 있다.
왜 필요한가	Nginx·Reverse Proxy·Load Balancer·Timeout 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Reverse proxy는 TLS 종료, 라우팅, 정적 파일, header 전달, load balancing을 담당할 수 있다. access log와 error log를 함께 보고 request_time과 upstream_response_time을 분리하면 병목 계층을 좁히기 쉽다.
선택 기준	proxy timeout을 무작정 크게 하면 장애를 감추고 socket/worker를 오래 점유한다. forwarded header는 신뢰할 proxy 경계에서만 믿는다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- reverse proxy와 upstream의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- proxy timeout을 무작정 크게 하면 장애를 감추고 socket/worker를 오래 점유한다. forwarded header는 신뢰할 proxy 경계에서만 믿는다.

실패 조건

- upstream port가 틀리거나 SELinux/권한/방화벽 문제면 application log에 요청이 없는데 502가 발생할 수 있다.

관찰

- DNS·connect·TLS·HTTP 단계별 시간
- status와 upstream log
- timeout budget

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D07 WORKED EXAMPLE

문제에서 검증까지

문제	사용자는 502를 받지만 애플리케이션에는 요청 로그가 없다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. DNS·connect·TLS·HTTP 단계별 시간 status와 upstream log timeout budget
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: upstream port가 틀리거나 SELinux/권한/방화벽 문제면 application log에 요청이 없는데 502가 발생할 수 있다.
조치	Nginx error log와 실제 upstream listen port를 확인하고 nginx -t 후 reload한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

사용자는 502를 받지만 애플리케이션에는 요청 로그가 없다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- /api proxy 구성
- upstream port 오류로 502

독립 단계

- 느린 backend로 504
- forwarded header 확인
- nginx -t와 curl smoke

명령

```
nginx -t
```

```
nginx -s reload
```

```
curl -i http://localhost/api/
```

```
tail -f /var/log/nginx/error.log
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- upstream port가 틀리거나 SELinux/권한/방화벽 문제면 application log에 요청이 없는데 502가 발생할 수 있다.

관찰 증거

- DNS-connect-TLS-HTTP 단계별 시간
- status와 upstream log
- timeout budget

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. reverse proxy의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Reverse proxy는 TLS 종료, 라우팅, 정적 파일, header 전달, load balancing을 담당할 수 있다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Nginx-Reverse Proxy-Load Balancer-Timeout 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · proxy timeout을 무작정 크게 하면 장애를 감추고 socket/worker를 오래 점유한다. forwarded header는 신뢰할 proxy 경계에서만 믿는다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · upstream port가 틀리거나 SELinux/권한/방화벽 문제면 application log에 요청이 없는데 502가 발생할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Nginx-Reverse Proxy-Load Balancer-Timeout을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 사용자는 502를 받지만 애플리케이션에는 요청 로그가 없다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 사용자는 502를 받지만 애플리케이션에는 요청 로그가 없다.을 재현한 뒤 Nginx error log와 실제 upstream listen port를 확인하고 nginx -t 후 reload한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Nginx Documentation](#) · official-primary

[Nginx Documentation](#) · official-primary

D08 · Spring IoC·DI·Bean Lifecycle

Spring IoC·DI·Bean Lifecycle 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 IoC의 정의와 내부 동작을 설명합니다.
- Spring IoC·DI·Bean Lifecycle에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: IoC, DI, ApplicationContext.
- Spring IoC·DI·Bean Lifecycle의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · IoC · DI · ApplicationContext · BeanDefinition · BeanPostProcessor · lifecycle · scope · constructor injection

Learn · 정의에서 관찰까지

정의	IoC는 객체 생성/조립/생명주기의 제어권을 container가 갖는 큰 원칙이고 DI는 의존성을 외부에서 제공하는 방식이다.
왜 필요한가	Spring IoC·DI·Bean Lifecycle 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	IoC는 객체 생성/조립/생명주기의 제어권을 container가 갖는 큰 원칙이고 DI는 의존성을 외부에서 제공하는 방식이다. ApplicationContext는 BeanDefinition을 읽고 bean을 만들며 후처리를 적용한다.
선택 기준	생성자 주입은 필수 의존성과 테스트를 명확히 하지만 순환참조가 보이면 주입 방식보다 책임 분리를 먼저 본다. singleton은 공유 인스턴스이므로 request state를 필드에 저장하지 않는다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- IoC와 DI의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 생성자 주입은 필수 의존성과 테스트를 명확히 하지만 순환참조가 보이면 주입 방식보다 책임 분리를 먼저 본다. singleton은 공유 인스턴스이므로 request state를 필드에 저장하지 않는다.

실패 조건

- singleton service의 mutable field는 동시 요청에서 값이 섞일 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D08 WORKED EXAMPLE

문제에서 검증까지

문제	currentUserId를 service field에 저장해 사용자 정보가 다른 요청과 섞인다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: singleton service의 mutable field는 동시 요청에서 값이 섞일 수 있다.
조치	method local/parameter로 상태를 이동하고 bean을 stateless하게 만든다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

currentUserId를 service field에 저장해 사용자 정보가 다른 요청과 섞인다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 생성자 주입 3개 bean
- lifecycle 로그

독립 단계

- BeanPostProcessor 로그
- mutable field 동시성 재현
- bean 목록 출력

명령

```
./gradlew bootRun
```

```
curl -s http://localhost:8080/lab/bean
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- singleton service의 mutable field는 동시 요청에서 값이 섞일 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. IoC의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · IoC는 객체 생성/조립/생명주기의 제어권을 container가 갖는 큰 원칙이고 DI는 의존성을 외부에서 제공하는 방식이다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Spring IoC·DI·Bean Lifecycle 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 생성자 주입은 필수 의존성과 테스트를 명확히 하지만 순환참조가 보이면 주입 방식보다 책임 분리를 먼저 본다. singleton 은 공유 인스턴스이므로 request state를 필드에 저장하지 않는다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · singleton service의 mutable field는 동시 요청에서 값이 섞일 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. Spring IoC·DI·Bean Lifecycle을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. currentUserId를 service field에 저장해 사용자 정보가 다른 요청과 섞인다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · currentUserId를 service field에 저장해 사용자 정보가 다른 요청과 섞인다.을 재현한 뒤 method local/parameter로 상태를 이동하고 bean을 stateless하게 만든다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Spring Framework Reference](#) · official-primary

D09 · Spring AOP·Proxy·MVC·Validation·Exception

Spring AOP·Proxy·MVC·Validation·Exception 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 AOP의 정의와 내부 동작을 설명합니다.
- Spring AOP·Proxy·MVC·Validation·Exception에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: AOP, proxy, advice.
- Spring AOP·Proxy·MVC·Validation·Exception의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · AOP · proxy · advice · pointcut · DispatcherServlet · HandlerMapping · validation · ControllerAdvice

Learn · 정의에서 관찰까지

정의	Spring AOP는 target 앞의 proxy가 method 호출을 가로채 공통 관심사를 적용한다.
왜 필요한가	Spring AOP·Proxy·MVC·Validation·Exception 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Spring AOP는 target 앞의 proxy가 method 호출을 가로채 공통 관심사를 적용한다. MVC는 DispatcherServlet에서 handler mapping/adapter, argument resolver, validation, message converter, exception resolver 를 거쳐 응답을 만든다.
선택 기준	AOP는 cross-cutting concern에 제한하고 business branching까지 숨기지 않는다. validation은 입력 형 식과 business rule의 책임 위치를 구분한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- AOP와 proxy의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- AOP는 cross-cutting concern에 제한하고 business branching까지 숨기지 않는다. validation은 입력 형 식과 business rule의 책임 위치를 구분한다.

실패 조건

- 내부 stack trace와 SQL 메시지를 client response에 노출하면 보안·운영 문제가 된다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	Controller에 validation, logging, transaction, business logic이 모두 들어있다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 내부 stack trace와 SQL 메시지를 client response에 노출하면 보안·운영 문제가 된다.
조치	Web 경계와 service 책임을 분리하고 ControllerAdvice로 안정적인 error contract를 만든다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

Controller에 validation, logging, transaction, business logic이 모두 들어있다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- @Around 실행시간 측정
- DTO validation 400

독립 단계

- domain exception 409
- 415 재현
- proxy class 출력

명령

```
./gradlew bootRun
```

```
curl -i -H "Content-Type: application/json" -d "{}" http://localhost:8080/orders
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 내부 stack trace와 SQL 메시지를 client response에 노출하면 보안·운영 문제가 된다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. AOP의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Spring AOP는 target 앞의 proxy가 method 호출을 가로채 공통 관심사를 적용한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Spring AOP·Proxy·MVC·Validation·Exception 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · AOP는 cross-cutting concern에 제한하고 business branching까지 숨기지 않는다. validation은 입력 형식과 business rule의 책임 위치를 구분한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 내부 stack trace와 SQL 메시지를 client response에 노출하면 보안·운영 문제가 된다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Spring AOP·Proxy·MVC·Validation·Exception을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Controller에 validation, logging, transaction, business logic이 모두 들어있다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · Controller에 validation, logging, transaction, business logic이 모두 들어있다.을 재현한 뒤 Web 경계와 service 책임을 분리하고 ControllerAdvice로 안정적인 error contract를 만든다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Spring Framework Reference](#) · official-primary

[Spring Framework Reference](#) · official-primary

D10 · Spring Transaction

Spring Transaction 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 TransactionInterceptor의 정의와 내부 동작을 설명합니다.
- Spring Transaction에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: TransactionInterceptor, PlatformTransactionManager, proxy boundary.
- Spring Transaction의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · TransactionInterceptor · PlatformTransactionManager · proxy boundary · REQUIRED · REQUIRES_NEW · rollback · readOnly

Learn · 정의에서 관찰까지

정의	@Transactional은 proxy가 호출을 가로채 TransactionInterceptor와 TransactionManager를 통해 transaction을 시작/참여/종료한다.
왜 필요한가	Spring Transaction 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	@Transactional은 proxy가 호출을 가로채 TransactionInterceptor와 TransactionManager를 통해 transaction을 시작/참여/종료한다. 기본 proxy mode에서 같은 객체의 self-invocation은 proxy를 통과하지 않아 interception이 빠질 수 있다.
선택 기준	REQUIRES_NEW는 독립 transaction을 만들지만 connection 증가와 outer rollback/inner commit 불일치를 낳을 수 있다. transaction 안의 느린 외부 I/O는 lock과 connection 점유시간을 늘린다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- TransactionInterceptor와 PlatformTransactionManager의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- REQUIRES_NEW는 독립 transaction을 만들지만 connection 증가와 outer rollback/inner commit 불일치를 낳을 수 있다. transaction 안의 느린 외부 I/O는 lock과 connection 점유시간을 늘린다.

실패 조건

- 예외를 catch하고 삼키면 transaction manager가 실패를 모른 채 commit할 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D10 WORKED EXAMPLE

문제에서 검증까지

문제	outer()가 같은 class의 @Transactional inner()를 호출해 예상한 rollback이 발생하지 않는다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그-metric-state 정상-경계-실패-재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 예외를 catch하고 삼키면 transaction manager가 실패를 모른 채 commit할 수 있다.
조치	별도 bean 경계로 책임을 분리하고 runtime/checked exception 및 propagation 결과를 테스트한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

outer()가 같은 class의 @Transactional inner()를 호출해 예상한 rollback이 발생하지 않는다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- self-invocation 재현
- 별도 bean 비교

독립 단계

- rollback 규칙 테스트
- REQUIRES_NEW 결과 비교
- transaction 안 slow I/O

명령

```
./gradlew test --tests "*Transaction*"
```

```
./gradlew bootRun
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 예외를 catch하고 삼키면 transaction manager가 실패를 모른 채 commit할 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. TransactionInterceptor의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · @Transactional은 proxy가 호출을 가로채 TransactionInterceptor와 TransactionManager를 통해 transaction 을 시작/참여/종료한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Spring Transaction 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · REQUIRES_NEW는 독립 transaction을 만들지만 connection 증가와 outer rollback/inner commit 불일치를 낳을 수 있다. transaction 안의 느린 외부 I/O는 lock과 connection 점유시간을 늘린다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 예외를 catch하고 삼키면 transaction manager가 실패를 모른 채 commit할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. Spring Transaction을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명 하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. outer()가 같은 class의 @Transactional inner()를 호출해 예상한 rollback이 발생하지 않는다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · outer()가 같은 class의 @Transactional inner()를 호출해 예상한 rollback이 발생하지 않는다.을 재현한 뒤 별도 bean 경계로 책임을 분리하고 runtime/checked exception 및 propagation 결과를 테스트한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Spring Framework Reference](#) · official-primary

[Spring Framework Reference](#) · official-primary

D11 · DB Transaction·Isolation·MVCC·Lock·Deadlock

DB Transaction·Isolation·MVCC·Lock·Deadlock 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 ACID의 정의와 내부 동작을 설명합니다.
- DB Transaction·Isolation·MVCC·Lock·Deadlock에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: ACID, dirty/non-repeatable/phantom read, MVCC.
- DB Transaction·Isolation·MVCC·Lock·Deadlock의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · ACID · dirty/non-repeatable/phantom read · MVCC · row lock · deadlock · optimistic locking

Learn · 정의에서 관찰까지

정의	Isolation은 동시성과 anomaly 사이의 trade-off이며 MVCC는 reader/writer 충돌을 줄이지만 write lock과 deadlock을 없애지 않는다.
왜 필요한가	DB Transaction·Isolation·MVCC·Lock·Deadlock 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Isolation은 동시성과 anomaly 사이의 trade-off이며 MVCC는 reader/writer 충돌을 줄이지만 write lock과 deadlock을 없애지 않는다. deadlock은 transaction들이 서로 가진 자원을 기다리는 cycle이다.
선택 기준	isolation을 높이면 정합성 보장은 늘 수 있지만 concurrency 비용이 커진다. 낙관적 lock은 version mismatch를 감지하고 충돌 처리 책임을 application에 둔다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- ACID와 dirty/non-repeatable/phantom read의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- isolation을 높이면 정합성 보장은 늘 수 있지만 concurrency 비용이 커진다. 낙관적 lock은 version mismatch를 감지하고 충돌 처리 책임을 application에 둔다.

실패 조건

- 오래 열린 transaction과 불일치한 lock 획득 순서는 blocking/deadlock을 만든다.

관찰

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D11 WORKED EXAMPLE

문제에서 검증까지

문제	T1은 A→B, T2는 B→A 순으로 update하여 cycle이 생긴다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 실행계획의 estimate와 실제 row lock wait와 transaction 경계 동일 입력의 결과 집합
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 오래 열린 transaction과 불일치한 lock 획득 순서는 blocking/deadlock을 만든다.
조치	lock 획득 순서를 통일하고 transaction을 짧게 하며 transient deadlock만 bounded retry한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

T1은 A→B, T2는 B→A 순으로 update하여 cycle이 생긴다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 두 session blocking
- 반대 순서 deadlock

독립 단계

- isolation 비교
- unique 경쟁
- version column 충돌

명령

```
SELECT ... FOR UPDATE
```

```
COMMIT
```

```
ROLLBACK
```

D11 VERIFY

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 오래 열린 transaction과 불일치한 lock 획득 순서는 blocking/deadlock을 만든다.

관찰 증거

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. ACID의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Isolation은 동시성과 anomaly 사이의 trade-off이며 MVCC는 reader/writer 충돌을 줄이지만 write lock과 deadlock을 없애지 않는다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. DB Transaction-Isolation-MVCC-Lock-Deadlock 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · isolation을 높이면 정합성 보장은 늘 수 있지만 concurrency 비용이 커진다. 낙관적 lock은 version mismatch를 감지 하고 충돌 처리 책임을 application에 둔다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 오래 열린 transaction과 불일치한 lock 획득 순서는 blocking/deadlock을 만든다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. DB Transaction·Isolation·MVCC·Lock·Deadlock을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. T1은 A→B, T2는 B→A 순으로 update하여 cycle이 생긴다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · T1은 A→B, T2는 B→A 순으로 update하여 cycle이 생긴다.을 재현한 뒤 lock 획득 순서를 통일하고 transaction을 짧게 하며 transient deadlock만 bounded retry한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Database 19c Documentation](#) · official-primary

[Official or Direct Reference](#) · official-primary

D12 · SQL·Index·Execution Plan

SQL·Index·Execution Plan 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 B-tree의 정의와 내부 동작을 설명합니다.
- SQL·Index·Execution Plan에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: B-tree, selectivity, cardinality.
- SQL·Index·Execution Plan의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · B-tree · selectivity · cardinality · range/full scan · nested loop · hash join · statistics · predicate

Learn · 정의에서 관찰까지

정의	B-tree index는 검색 범위를 줄이지만 결과 비율이 높거나 table access가 많으면 full scan이 더 유리할 수 있다.
왜 필요한가	SQL·Index·Execution Plan 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	B-tree index는 검색 범위를 줄이지만 결과 비율이 높거나 table access가 많으면 full scan이 더 유리할 수 있다. 실행계획은 optimizer의 operator tree이고 예상 rows와 실제 rows의 차이는 plan 선택 오류의 중요한 단서다.
선택 기준	인덱스는 write/storage 비용이 있고 복합 인덱스는 column 순서와 predicate 형태가 중요하다. hint로 계획 을 고정하기 전에 통계와 data skew를 확인한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- B-tree와 selectivity의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 인덱스는 write/storage 비용이 있고 복합 인덱스는 column 순서와 predicate 형태가 중요하다. hint로 계획 을 고정하기 전에 통계와 data skew를 확인한다.

실패 조건

- 함수/형변환 때문에 예상한 index access가 사라지거나 cardinality estimate가 틀리면 잘못된 join method가 선택될 수 있다.

관찰

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D12 WORKED EXAMPLE

문제에서 검증까지

문제	기간 조회가 느려 인덱스를 추가했지만 결과가 table의 큰 비율이라 random access만 증가한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 실행계획의 estimate와 실제 row lock wait와 transaction 경계 동일 입력의 결과 집합
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 함수/형변환 때문에 예상한 index access가 사라지거나 cardinality estimate가 틀리면 잘못된 join method가 선택될 수 있다.
조치	결과 건수/선택도/plan을 확인하고 query 범위, partition, 적절한 index 또는 full scan을 비교한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

기간 조회가 느려 인덱스를 추가했지만 결과가 table의 큰 비율이라 random access만 증가한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 대량 sample table
- index 전후 plan

독립 단계

- 복합 인덱스 순서
- 함수 predicate 비교
- join 방식 비교

명령

```
EXPLAIN PLAN FOR SELECT ...;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 함수/형변환 때문에 예상한 index access가 사라지거나 cardinality estimate가 틀리면 잘못된 join method가 선택될 수 있다.

관찰 증거

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. B-tree의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · B-tree index는 검색 범위를 줄이지만 결과 비율이 높거나 table access가 많으면 full scan이 더 유리할 수 있다. 정의 뒤 에 실제 목적과 경계를 붙여 설명합니다.

2. SQL-Index-Execution Plan 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 인덱스는 write/storage 비용이 있고 복합 인덱스는 column 순서와 predicate 형태가 중요하다. hint로 계획을 고정하기 전에 통계와 data skew를 확인한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 함수/형변환 때문에 예상한 index access가 사라지거나 cardinality estimate가 틀리면 잘못된 join method가 선택될 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. SQL·Index·Execution Plan을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서 로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. SQL·Index·Execution Plan에서 실행계획과 실제 실행 데이터를 어떤 순서로 비교할지 SQL/관측 절차를 작성하세요.

유형 sql · 난이도 medium

기준 답안 · 기간 조회가 느려 인덱스를 추가했지만 결과가 table의 큰 비율이라 random access만 증가한다.을 재현한 뒤 결과 건수/선택도/plan을 확인하고 query 범위, partition, 적절한 index 또는 full scan을 비교한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Database 19c Documentation](#) · official-primary

[Oracle Database 19c Documentation](#) · official-primary

D13 · Oracle Tuning·Partition

Oracle Tuning·Partition 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 partition의 정의와 내부 동작을 설명합니다.
- Oracle Tuning·Partition에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: partition, pruning, local/global index.
- Oracle Tuning·Partition의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · partition · pruning · local/global index · bind · histogram · DBMS_XPLAN · bulk DML

Learn · 정의에서 관찰까지

정의	Partition은 큰 table의 성능뿐 아니라 lifecycle·관리성·가용성을 나누는 도구다.
왜 필요한가	Oracle Tuning·Partition 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Partition은 큰 table의 성능뿐 아니라 lifecycle·관리성·가용성을 나누는 도구다. partition key와 predicate가 맞으면 pruning으로 불필요한 partition을 제외할 수 있으며 DBMS_XPLAN의 PSTART/PSTOP 같은 단서를 확인한다.
선택 기준	시간 기반 데이터에서 row-by-row DELETE 대신 partition 단위 정리가 유리할 수 있지만 FK/index/archive/운영정책을 함께 본다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- partition와 pruning의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 시간 기반 데이터에서 row-by-row DELETE 대신 partition 단위 정리가 유리할 수 있지만 FK/index/archive/운영정책을 함께 본다.

실패 조건

- partition key 가공으로 pruning이 깨지거나 global index maintenance 문제가 생길 수 있다.

관찰

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D13 WORKED EXAMPLE

문제에서 검증까지

문제	오래된 시간 데이터를 대량 DELETE하면서 lock/undo 부하가 증가한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 실행계획의 estimate와 실제 row lock wait와 transaction 경계 동일 입력의 결과 집합
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: partition key 가공으로 pruning이 깨지거나 global index maintenance 문제가 생길 수 있다.
조치	시간 partition과 lifecycle이 맞는지 검토하고 partition 단위 정리로 변경한 뒤 plan과 정합성을 검증한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

오래된 시간 데이터를 대량 DELETE하면서 lock/undo 부하가 증가한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- range partition 생성
- pruning plan 확인

독립 단계

- 가공 predicate 비교
- DELETE와 partition 정리 비교
- bind skew 학습

명령

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL,NULL,'ALLSTATS LAST'));
```

D13 VERIFY

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- partition key 가공으로 pruning이 깨지거나 global index maintenance 문제가 생길 수 있다.

관찰 증거

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. partition의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Partition은 큰 table의 성능뿐 아니라 lifecycle-관리성·가용성을 나누는 도구다. 정의 뒤에 실제 목적과 경계를 붙여 설 명합니다.

2. Oracle Tuning-Partition 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 시간 기반 데이터에서 row-by-row DELETE 대신 partition 단위 정리가 유리할 수 있지만 FK/index/archive/운영정책을 함께 본다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · partition key 가공으로 pruning이 깨지거나 global index maintenance 문제가 생길 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. Oracle Tuning-Partition을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Oracle Tuning-Partition에서 실행계획과 실제 실행 데이터를 어떤 순서로 비교할지 SQL/관측 절차를 작성하세요.

유형 sql · 난이도 medium

기준 답안 · 오래된 시간 데이터를 대량 DELETE하면서 lock/undo 부하가 증가한다.을 재현한 뒤 시간 partition과 lifecycle이 맞는지 검토하고 partition 단위 정리로 변경한 뒤 plan과 정합성을 검증한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Database 19c Documentation](#) · official-primary

[Oracle Database 19c Documentation](#) · official-primary

CHAPTER 08 · Spring Batch와 대외 연계

D14 · Spring Batch

Spring Batch 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 Job의 정의와 내부 동작을 설명합니다.
- Spring Batch에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: Job, Step, Chunk.
- Spring Batch의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · Job · Step · Chunk · JobInstance · JobExecution · StepExecution · ExecutionContext · restart · retry · skip

Learn · 정의에서 관찰까지

정의	JobInstance는 논리 실행, JobExecution은 그 instance의 실제 시도이며 Chunk는 read/process/write를 commit interval 단 위 transaction으로 처리한다.
왜 필요한가	Spring Batch 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	JobInstance는 논리 실행, JobExecution은 그 instance의 실제 시도이며 Chunk는 read/process/write를 commit interval 단 위 transaction으로 처리한다. restart/retry/skip은 서로 다른 복구 개념이다.
선택 기준	chunk size를 키우면 round trip은 줄 수 있지만 transaction log, memory, rollback 범위가 커질 수 있다. ExecutionContext는 restart state이지 큰 business data 저장소가 아니다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- Job와 Step의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- chunk size를 키우면 round trip은 줄 수 있지만 transaction log, memory, rollback 범위가 커질 수 있다. ExecutionContext는 restart state이지 큰 business data 저장소가 아니다.

실패 조건

- retry 가능한 transient 오류와 deterministic 오류를 구분하지 않으면 무한/불필요 재시도가 생긴다.

관찰

- JobInstance·Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D14 WORKED EXAMPLE

문제에서 검증까지

문제	10건을 5건 chunk로 처리하다 7번째 item이 한 번 실패한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. JobInstance-Execution 상태 처리 건수와 기준일 부분 실패-재처리 기록
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: retry 가능한 transient 오류와 deterministic 오류를 구분하지 않으면 무한/불필요 재시도가 생긴다.
조치	두 번째 chunk rollback→retry→재시작 metadata를 관찰하고 writer 중복 안전성을 확인한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

10건을 5건 chunk로 처리하다 7번째 item이 한 번 실패한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 10건 5개 chunk
- transient retry

독립 단계

- 항상 실패 exhaustion
- process restart
- JobParameters에 따른 Instance 비교

명령

```
./gradlew bootRun --args="run.id=1 businessDate=2026-08-26"
```


실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- retry 가능한 transient 오류와 deterministic 오류를 구분하지 않으면 무한/불필요 재시도가 생긴다.

관찰 증거

- JobInstance-Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. Job의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · JobInstance는 논리 실행, JobExecution은 그 instance의 실제 시도이며 Chunk는 read/process/write를 commit interval 단위 transaction으로 처리한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Spring Batch 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · chunk size를 키우면 round trip은 줄 수 있지만 transaction log, memory, rollback 범위가 커질 수 있다. ExecutionContext는 restart state이지 큰 business data 저장소가 아니다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · retry 가능한 transient 오류와 deterministic 오류를 구분하지 않으면 무한/불필요 재시도가 생긴다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. Spring Batch을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 10건을 5건 chunk로 처리하다 7번째 item이 한 번 실패한다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 10건을 5건 chunk로 처리하다 7번째 item이 한 번 실패한다.을 재현한 뒤 두 번째 chunk rollback→retry→재시작 metadata를 관찰하고 writer 중복 안전성을 확인한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Spring Batch Reference](#) · official-primary

[Spring Batch Reference](#) · official-primary

[Spring Batch Reference](#) · official-primary

D15 · EAI·File 연계·Control-M·정산 Batch

EAI·File 연계·Control-M·정산 Batch 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 EAI의 정의와 내부 동작을 설명합니다.
- EAI·File 연계·Control-M·정산 Batch에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: EAI, File-to-File, interface table.
- EAI·File 연계·Control-M·정산 Batch의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · EAI · File-to-File · interface table · polling · predecessor/successor · event/condition · reconciliation · rerun

Learn · 정의에서 관찰까지

정의	대외 연계는 접속 성공→수신→형식/중복 검증→business 반영→후속 batch→송신→상대 확인 단계로 본다.
왜 필요한가	EAI·File 연계·Control-M·정산 Batch 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	대외 연계는 접속 성공→수신→형식/중복 검증→business 반영→후속 batch→송신→상대 확인 단계로 본다. scheduler의 성공 상태 와 business 데이터 정합성은 별도다.
선택 기준	DB-Link를 interface table/polling으로 분리하면 재처리와 상태 관리가 쉬워지지만 poison record, polling interval, retention 운영이 추가된다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- EAI와 File-to-File의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- DB-Link를 interface table/polling으로 분리하면 재처리와 상태 관리가 쉬워지지만 poison record, polling interval, retention 운영이 추가된다.

실패 조건

- partial file을 읽거나 원천 batch를 다시 돌려 이미 완료된 외부 송신까지 중복할 수 있다.

관찰

- JobInstance·Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D15 WORKED EXAMPLE

문제에서 검증까지

문제	원천 batch는 성공했지만 외부 송신만 실패했다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. JobInstance-Execution 상태 처리 건수와 기준일 부분 실패-재처리 기록
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: partial file을 읽거나 원천 batch를 다시 돌려 이미 완료된 외부 송신까지 중복할 수 있다.
조치	업무 결과와 delivery 상태를 분리하고 failed delivery만 같은 key로 재시도한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

원천 batch는 성공했지만 외부 송신만 실패했다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB-파일-배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- CSV staging/validation
- partial file과 atomic rename

독립 단계

- interface table 상태 machine
- scheduler flow 모사
- 기준일 재실행/대사

명령

```
sha256sum sample.csv
```

```
wc -l sample.csv
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- partial file을 읽거나 원천 batch를 다시 돌려 이미 완료된 외부 송신까지 중복할 수 있다.

관찰 증거

- JobInstance-Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. EAI의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 대외 연계는 접속 성공→수신→형식/중복 검증→business 반영→후속 batch→송신→상대 확인 단계로 본다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. EAI-File 연계·Control-M·정산 Batch 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · DB-Link를 interface table/polling으로 분리하면 재처리와 상태 관리가 쉬워지지만 poison record, polling interval, retention 운영이 추가된다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · partial file을 읽거나 원천 batch를 다시 돌려 이미 완료된 외부 송신까지 중복할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. EAI-File 연계-Control-M-정산 Batch을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 원천 batch는 성공했지만 외부 송신만 실패했다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 medium

기준 답안 · 원천 batch는 성공했지만 외부 송신만 실패했다.을 재현한 뒤 업무 결과와 delivery 상태를 분리하고 failed delivery만 같은 key로 재시도한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[BMC Control-M Documentation](#) · official-primary

[BMC Control-M Documentation](#) · official-primary

D16 · Array·String·Hash·Two Pointer

Array·String·Hash·Two Pointer 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 frequency map의 정의와 내부 동작을 설명합니다.
- Array·String·Hash·Two Pointer에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: frequency map, two pointer, sliding window.
- Array·String·Hash·Two Pointer의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · frequency map · two pointer · sliding window · invariant · edge case · overflow

Learn · 정의에서 관찰까지

정의	문제를 코딩하기 전에 N, 정렬 여부, 중복, 음수, 결과 범위를 읽고 자료구조 후보를 정한다.
왜 필요한가	Array·String·Hash·Two Pointer 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	문제를 코딩하기 전에 N, 정렬 여부, 중복, 음수, 결과 범위를 읽고 자료구조 후보를 정한다. two pointer/sliding window는 pointer 이동에 사용할 단조성 또는 유지 가능한 invariant가 있을 때 적합하다.
선택 기준	영문 소문자처럼 key 범위가 고정되면 배열이 Map보다 단순할 수 있고, 입력이 정렬되지 않았으면 정렬 비용과 HashMap 대안을 비교한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- frequency map와 two pointer의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 영문 소문자처럼 key 범위가 고정되면 배열이 Map보다 단순할 수 있고, 입력이 정렬되지 않았으면 정렬 비용과 HashMap 대안을 비교한다.

실패 조건

- 음수 때문에 window sum이 단조롭지 않으면 단순 sliding window가 잘못된 해를 낼 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D16 WORKED EXAMPLE

문제에서 검증까지

문제	정렬된 배열에서 target 합 pair를 찾는다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 음수 때문에 window sum이 단조롭지 않으면 단순 sliding window가 잘못된 해를 낼 수 있다.
조치	$O(n^2)$ brute force를 oracle로 두고 $O(n)$ two pointer 결과를 random input으로 비교한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

정렬된 배열에서 target 합 pair를 찾는다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 빈도 집계
- two-sum

독립 단계

- 고정/가변 window
- brute force oracle
- edge case/overflow

명령

```
./gradlew test --tests "*Day16"
```

D16 VERIFY

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 음수 때문에 window sum이 단조롭지 않으면 단순 sliding window가 잘못된 해를 낼 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. frequency map의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 문제를 코딩하기 전에 N, 정렬 여부, 중복, 음수, 결과 범위를 읽고 자료구조 후보를 정한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Array·String·Hash·Two Pointer 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 영문 소문자처럼 key 범위가 고정되면 배열이 Map보다 단순할 수 있고, 입력이 정렬되지 않았으면 정렬 비용과 HashMap 대안을 비교한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 음수 때문에 window sum이 단조롭지 않으면 단순 sliding window가 잘못된 해를 낼 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. Array-String-Hash-Two Pointer을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Array-String-Hash-Two Pointer의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 정렬된 배열에서 target 합 pair를 찾는다.을 재현한 뒤 $O(n^2)$ brute force를 oracle로 두고 $O(n)$ two pointer 결과를 random input으로 비교한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

D17 · Stack·Queue·Heap·Sort

Stack·Queue·Heap·Sort 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 Deque의 정의와 내부 동작을 설명합니다.
- Stack·Queue·Heap·Sort에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: Deque, stack, queue.
- Stack·Queue·Heap·Sort의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · Deque · stack · queue · PriorityQueue · heap · comparator · stable sort

Learn · 정의에서 관찰까지

정의	Deque는 stack과 queue를 모두 표현할 수 있고 PriorityQueue는 최우선 원소를 빠르게 제공하지만 전체 요소 정렬을 보장하지 않는다.
왜 필요한가	Stack·Queue·Heap·Sort 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Deque는 stack과 queue를 모두 표현할 수 있고 PriorityQueue는 최우선 원소를 빠르게 제공하지만 전체 요소 정렬을 보장하지 않는다. Comparator는 overflow 없이 total order를 만들어야 한다.
선택 기준	상위 K처럼 K가 N보다 작고 streaming 유지가 필요하다면 heap이 유리하지만 한 번 전체 결과가 필요하다면 sort 가 더 단순할 수 있다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- Deque와 stack의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 상위 K처럼 K가 N보다 작고 streaming 유지가 필요하다면 heap이 유리하지만 한 번 전체 결과가 필요하다면 sort 가 더 단순할 수 있다.

실패 조건

- Comparator에서 a-b를 반환하면 integer overflow로 순서가 뒤집힐 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D17 WORKED EXAMPLE

문제에서 검증까지

문제	실시간 점수 상위 K개를 유지한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: Comparator에서 a-b를 반환하면 integer overflow로 순서가 뒤집힐 수 있다.
조치	K 크기의 min-heap을 유지하고 full sort와 N/K 변화에서 선택 기준을 비교한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

실시간 점수 상위 K개를 유지한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 괄호/stack
- BFS queue

독립 단계

- top-K heap
- Comparator overflow
- full sort vs heap

명령

```
./gradlew test --tests "*Day17"
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- Comparator에서 a-b를 반환하면 integer overflow로 순서가 뒤집힐 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. Deque의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Deque는 stack과 queue를 모두 표현할 수 있고 PriorityQueue는 최우선 원소를 빠르게 제공하지만 전체 요소 정렬을 보장하지 않는다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Stack·Queue·Heap·Sort 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 상위 K처럼 K가 N보다 작고 streaming 유지가 필요하다면 heap이 유리하지만 한 번 전체 결과가 필요하다면 sort가 더 단순 할 수 있다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · Comparator에서 a-b를 반환하면 integer overflow로 순서가 뒤집힐 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. Stack-Queue-Heap-Sort을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서 로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Stack-Queue-Heap-Sort의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 실시간 점수 상위 K개를 유지한다.을 재현한 뒤 K 크기의 min-heap을 유지하고 full sort와 N/K 변화에서 선택 기준을 비교한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

D18 · Binary Search·Greedy·Prefix·Interval

Binary Search·Greedy·Prefix·Interval 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 monotonic predicate의 정의와 내부 동작을 설명합니다.
- Binary Search·Greedy·Prefix·Interval에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: monotonic predicate, lower/upper bound, greedy proof.
- Binary Search·Greedy·Prefix·Interval의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · monotonic predicate · lower/upper bound · greedy proof · prefix sum · interval

Learn · 정의에서 관찰까지

정의	Binary search의 핵심은 정답 후보에 대한 predicate가 단조로운지 확인하는 것이다.
왜 필요한가	Binary Search·Greedy·Prefix·Interval 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Binary search의 핵심은 정답 후보에 대한 predicate가 단조로운지 확인하는 것이다. Greedy는 현재 선택이 전역 최적해로 이어지는 구조적 근거가 필요하고 Prefix sum은 전처리로 반복 range query를 빠르게 한다.
선택 기준	Binary search boundary는 inclusive/exclusive 규칙을 하나로 고정한다. Greedy 근거가 없으면 DP 후보를 비교한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- monotonic predicate와 lower/upper bound의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- Binary search boundary는 inclusive/exclusive 규칙을 하나로 고정한다. Greedy 근거가 없으면 DP 후보를 비교한다.

실패 조건

- off-by-one과 잘못된 greedy 선택은 작은 입력에서는 통과하고 큰/특수 입력에서만 실패하기 쉽다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D18 WORKED EXAMPLE

문제에서 검증까지

문제	D일 안에 작업을 끝낼 최소 capacity를 구한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: off-by-one과 잘못된 greedy 선택은 작은 입력에서는 통과하고 큰/특수 입력에서만 실패하기 쉽다.
조치	canFinish(capacity)의 단조성을 이용해 answer-space lower-bound search를 구현한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

D일 안에 작업을 끝낼 최소 capacity를 구한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- lower bound
- answer-space search

독립 단계

- interval greedy
- prefix sum
- brute force random 검증

명령

```
./gradlew test --tests "*Day18"
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- off-by-one과 잘못된 greedy 선택은 작은 입력에서는 통과하고 큰/특수 입력에서만 실패하기 쉽다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. monotonic predicate의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Binary search의 핵심은 정답 후보에 대한 predicate가 단조로운지 확인하는 것이다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Binary Search-Greedy-Prefix-Interval 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · Binary search boundary는 inclusive/exclusive 규칙을 하나로 고정한다. Greedy 근거가 없으면 DP 후보를 비교한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · off-by-one과 잘못된 greedy 선택은 작은 입력에서는 통과하고 큰/특수 입력에서만 실패하기 쉽다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. Binary Search-Greedy-Prefix-Interval을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Binary Search-Greedy-Prefix-Interval의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · D일 안에 작업을 끝낼 최소 capacity를 구한다.을 재현한 뒤 canFinish(capacity)의 단조성을 이용해 answer-space lower-bound search를 구현한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

D19 · BFS·DFS·Graph·Union-Find

BFS·DFS·Graph·Union-Find 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 vertex의 정의와 내부 동작을 설명합니다.
- BFS·DFS·Graph·Union-Find에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: vertex, edge, adjacency list.
- BFS·DFS·Graph·Union-Find의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · vertex · edge · adjacency list · BFS · DFS · visited · Union-Find · path compression

Learn · 정의에서 관찰까지

정의	그래프 문제는 알고리즘보다
왜 필요한가	BFS·DFS·Graph·Union-Find 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	그래프 문제는 알고리즘보다 vertex/edge 모델링을 먼저 한다. BFS는 unweighted shortest path에 강하고 DFS는 경로/구성요 소/백트래킹에 자주 쓰며 Union-Find는 연결 집합 대표를 관리한다.
선택 기준	DFS recursion은 깊은 graph에서 stack 위험이 있고 Union-Find는 실제 경로를 제공하지 않는다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- vertex와 edge의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- DFS recursion은 깊은 graph에서 stack 위험이 있고 Union-Find는 실제 경로를 제공하지 않는다.

실패 조건

- BFS에서 dequeue 시점에만 visited를 표시하면 같은 node가 queue에 여러 번 들어갈 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D19 WORKED EXAMPLE

문제에서 검증까지

문제	서버 연결 정보에서 component 수와 동적 연결 여부를 구한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: BFS에서 dequeue 시점에만 visited를 표시하면 같은 node가 queue에 여러 번 들어갈 수 있다.
조치	일회성 전체 graph는 BFS/DFS, 반복 connectivity query는 Union-Find로 비교한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

서버 연결 정보에서 component 수와 동적 연결 여부를 구한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- grid BFS
- components DFS/BFS

독립 단계

- deep DFS iterative
- Union-Find 최적화
- dependency cycle

명령

```
./gradlew test --tests "*Day19*"
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- BFS에서 dequeue 시점에만 visited를 표시하면 같은 node가 queue에 여러 번 들어갈 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. vertex의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 그래프 문제는 알고리즘보다 vertex/edge 모델링을 먼저 한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. BFS·DFS·Graph-Union-Find 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · DFS recursion은 깊은 graph에서 stack 위험이 있고 Union-Find는 실제 경로를 제공하지 않는다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · BFS에서 dequeue 시점에만 visited를 표시하면 같은 node가 queue에 여러 번 들어갈 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. BFS·DFS·Graph·Union·Find(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순 서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. BFS·DFS·Graph·Union·Find의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 서버 연결 정보에서 component 수와 동적 연결 여부를 구한다.을 재현한 뒤 일회성 전체 graph는 BFS/DFS, 반복 connectivity query는 Union-Find로 비교한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

CHAPTER 13 · DP·Backtracking

D20 · DP·Backtracking·Brute Force

DP·Backtracking·Brute Force 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 state의 정의와 내부 동작을 설명합니다.
- DP·Backtracking·Brute Force에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: state, transition, base case.
- DP·Backtracking·Brute Force의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · state · transition · base case · memoization · tabulation · backtracking · pruning

Learn · 정의에서 관찰까지

정의	DP는 중복되는 부분 문제의 state를 정의하고 결과를 재사용한다.
왜 필요한가	DP·Backtracking·Brute Force 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	DP는 중복되는 부분 문제의 state를 정의하고 결과를 재사용한다. 먼저 $dp[i]$ 의 의미를 한 문장으로 쓰고 base/transition/iteration order를 정한다. Backtracking은 선택→검사→복구를 반복하며 안전한 pruning 근거가 필요하다.
선택 기준	Memoization과 tabulation은 접근하는 state와 recursion depth에 따라 선택한다. greedy 근거가 없으면 DP를 검토한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- state와 transition의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- Memoization과 tabulation은 접근하는 state와 recursion depth에 따라 선택한다. greedy 근거가 없으면 DP를 검토한다.

실패 조건

- 정답 근거 없이 pruning하면 필요한 branch를 제거해 오답이 된다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	일반 coin set의 최소 동전 수를 구한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 정답 근거 없이 pruning하면 필요한 branch를 제거해 오답이 된다.
조치	dp[x]를 정의하고 brute force 작은 입력으로 DP 결과를 검증한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

일반 coin set의 최소 동전 수를 구한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- coin DP
- memo/tabulation 비교

독립 단계

- N-Queens pruning
- brute force oracle
- 2D→1D memory 줄이기

명령

```
./gradlew test --tests "*Day20"
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 정답 근거 없이 pruning하면 필요한 branch를 제거해 오답이 된다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. state의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · DP는 중복되는 부분 문제의 state를 정의하고 결과를 재사용한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. DP·Backtracking·Brute Force 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · Memoization과 tabulation은 접근하는 state와 recursion depth에 따라 선택한다. greedy 근거가 없으면 DP를 검토 한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 정답 근거 없이 pruning하면 필요한 branch를 제거해 오답이 된다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. DP-Backtracking-Brute Force을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. DP-Backtracking-Brute Force의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 일반 coin set의 최소 동전 수를 구한다.을 재현한 뒤 dp[x]를 정의하고 brute force 작은 입력으로 DP 결과를 검증한다. 조치를 적용하고 정상:경계:실패:재실행 증거를 비교해야 한다.

공식 자료

[Oracle Java SE 21 Documentation](#) · official-primary

D21 · SQL 심화와 Java 구현 품질

SQL 심화와 Java 구현 품질 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 window function의 정의와 내부 동작을 설명합니다.
- SQL 심화와 Java 구현 품질에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: window function, ROW_NUMBER, NULL.
- SQL 심화와 Java 구현 품질의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · window function · ROW_NUMBER · NULL · 3-valued logic · CTE · outer join · readability · testability

Learn · 정의에서 관찰까지

정의	Window function은 row를 축약하지 않고 partition 기준 순위/집계를 붙인다.
왜 필요한가	SQL 심화와 Java 구현 품질 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Window function은 row를 축약하지 않고 partition 기준 순위/집계를 붙인다. SQL NULL은 unknown semantics를 가져 = NULL 대신 IS NULL을 사용하고 NOT IN에 NULL이 섞이면 결과가 달라질 수 있다.
선택 기준	CTE는 가독성을 높이지만 DB/version별 optimizer 처리 차이를 확인한다. Java 구현도 naming, side effect, null/exception, test boundary를 함께 본다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- window function과 ROW_NUMBER의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- CTE는 가독성을 높이지만 DB/version별 optimizer 처리 차이를 확인한다. Java 구현도 naming, side effect, null/exception, test boundary를 함께 본다.

실패 조건

- outer join 후 WHERE 조건을 잘못 두면 사실상 inner join처럼 결과가 줄어들 수 있다.

관찰

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D21 WORKED EXAMPLE

문제에서 검증까지

문제	고객별 최근 주문 1건을 반복 correlated subquery로 찾는다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 실행계획의 estimate와 실제 row lock wait와 transaction 경계 동일 입력의 결과 집합
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: outer join 후 WHERE 조건을 잘못 두면 사실상 inner join처럼 결과가 줄어들 수 있다.
조치	ROW_NUMBER OVER(PARTITION BY customer ORDER BY created_at DESC)로 전환하고 plan/index를 확인한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

고객별 최근 주문 1건을 반복 correlated subquery로 찾는다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- ROW_NUMBER/RANK
- NOT IN vs NOT EXISTS

독립 단계

- outer join ON/WHERE
- Java refactor
- SQL3+Java2 self-review

명령

```
./gradlew test
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- outer join 후 WHERE 조건을 잘못 두면 사실상 inner join처럼 결과가 줄어 들 수 있다.

관찰 증거

- 실행계획의 estimate와 실제 row
- lock wait와 transaction 경계
- 동일 입력의 결과 집합

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. window function의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Window function은 row를 축약하지 않고 partition 기준 순위/집계를 붙인다. 정의 뒤에 실제 목적과 경계를 붙여 설명 합니다.

2. SQL 심화와 Java 구현 품질에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · CTE는 가독성을 높이지만 DB/version별 optimizer 처리 차이를 확인한다. Java 구현도 naming, side effect, null/exception, test boundary를 함께 본다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · outer join 후 WHERE 조건을 잘못 두면 사실상 inner join처럼 결과가 줄어 들 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. SQL 심화와 Java 구현 품질을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. SQL 심화와 Java 구현 품질에서 실행계획과 실제 실행 데이터를 어떤 순서로 비교할지 SQL/관측 절차를 작성하세요.

유형 sql · 난이도 medium

기준 답안 · 고객별 최근 주문 1건을 반복 correlated subquery로 찾는다.을 재현한 뒤 ROW_NUMBER OVER(PARTITION BY customer ORDER BY created_at DESC)로 전환하고 plan/index를 확인한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Oracle Database 19c Documentation](#) · official-primary

D22 · 통합 코딩 실습

통합 코딩 실습 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 timebox의 정의와 내부 동작을 설명합니다.
- 통합 코딩 실습에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: timebox, constraints, brute force oracle.
- 통합 코딩 실습의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · timebox · constraints · brute force oracle · edge case · postmortem · complexity

Learn · 정의에서 관찰까지

정의	제한시간에서도 5분 요구사항/제약 분석→자료구조/브루트포스 후보→구현→edge case→복잡도 설명 순서를 유지한다.
왜 필요한가	통합 코딩 실습 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	제한시간에서도 5분 요구사항/제약 분석→자료구조/브루트포스 후보→구현→edge case→복잡도 설명 순서를 유지한다. 오답은 패턴 이름보다 사고 단계의 실패로 분류한다.
선택 기준	정답을 본 직후 베끼는 대신 잠시 뒤나 다음 날 다시 구현해 retrieval을 만든다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- timebox와 constraints의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 정답을 본 직후 베끼는 대신 잠시 뒤나 다음 날 다시 구현해 retrieval을 만든다.

실패 조건

- 문제 해석 없이 코딩부터 시작하면 알고리즘 지식이 있어도 시간을 잃는다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D22 WORKED EXAMPLE

문제에서 검증까지

문제	90분 세트에서 마지막 문제의 해석에 25분을 사용한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그-metric-state 정상-경계-실패-재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 문제 해석 없이 코딩부터 시작하면 알고리즘 지식이 있어도 시간을 잃는다.
조치	다음 세트부터 첫 5분 분석 메모를 강제하고 반복 실패 유형 감소를 추적한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

90분 세트에서 마지막 문제의 해석에 25분을 사용한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- Java3+SQL2 120분
- 문제별 5분 분석

독립 단계

- brute force 검증
- 오답 분류
- 24시간 뒤 재풀이

명령

```
./gradlew test
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 문제 해석 없이 코딩부터 시작하면 알고리즘 지식이 있어도 시간을 잃는다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. timebox의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 제한시간에서도 5분 요구사항/제약 분석→자료구조/브루트포스 후보→구현→edge case→복잡도 설명 순서를 유지한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. 통합 코딩 실습에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 정답을 본 직후 베끼는 대신 잠시 뒤나 다음 날 다시 구현해 retrieval을 만든다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 문제 해석 없이 코딩부터 시작하면 알고리즘 지식이 있어도 시간을 잃는다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. 통합 코딩 실습을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 통합 코딩 실습의 실패 조건을 최소 코드로 재현하고 경계 입력을 검증하는 방법을 작성하세요.

유형 code · 난이도 medium

기준 답안 · 90분 세트에서 마지막 문제의 해석에 25분을 사용한다.을 재현한 뒤 다음 세트부터 첫 5분 분석 메모를 강제하고 반복 실패 유형 감소를 추적한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

Retrieval and Spacing Research · peer-reviewed-direct

D23 · Timeout·Retry·Backoff·Circuit Breaker

Timeout·Retry·Backoff·Circuit Breaker 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 deadline의 정의와 내부 동작을 설명합니다.
- Timeout·Retry·Backoff·Circuit Breaker에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: deadline, timeout budget, retry.
- Timeout·Retry·Backoff·Circuit Breaker의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · deadline · timeout budget · retry · exponential backoff · jitter · retry storm · breaker · bulkhead

Learn · 정의에서 관찰까지

정의	분산 호출은 부분 실패하므로 전체 deadline 안에 connect/acquire/read/retry budget을 배분한다.
왜 필요한가	Timeout·Retry·Backoff·Circuit Breaker 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	분산 호출은 부분 실패하므로 전체 deadline 안에 connect/acquire/read/retry budget을 배분한다. Retry는 transient 오류에 제한 하고 backoff+jitter로 재폭주를 줄이며 circuit breaker는 일정 기간 호출을 차단해 downstream을 보호한다.
선택 기준	retry와 breaker의 목적은 다르고 여러 계층이 각자 재시도하면 횟수가 곱셈으로 폭증할 수 있다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- deadline와 timeout budget의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- retry와 breaker의 목적은 다르고 여러 계층이 각자 재시도하면 횟수가 곱셈으로 폭증할 수 있다.

실패 조건

- deterministic 4xx까지 재시도하거나 write idempotency 없이 retry하면 부하와 중복 부수효과가 커진다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D23 WORKED EXAMPLE

문제에서 검증까지

문제	A→B→C 각 계층이 3회 retry하여 C 장애 시 요청이 폭증한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: deterministic 4xx까지 재시도하거나 write idempotency 없이 retry하면 부하와 중복 부수효과가 커진다.
조치	retry ownership을 한 경계로 제한하고 backoff+jitter, breaker, deadline을 적용한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

A→B→C 각 계층이 3회 retry하여 C 장애 시 요청이 폭증한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 지연/실패 downstream
- connect/read timeout

독립 단계

- fixed vs jitter retry
- POST idempotency
- breaker state 모사

명령

```
curl --connect-timeout 1 --max-time 3 ...
```

```
./gradlew test --tests "*Retry*"
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- deterministic 4xx까지 재시도하거나 write idempotency 없이 retry하면 부하와 중복 부수효과가 커진다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. deadline의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 분산 호출은 부분 실패하므로 전체 deadline 안에 connect/acquire/read/retry budget을 배분한다. 정의 뒤에 실제 목 적과 경계를 붙여 설명합니다.

2. Timeout-Retry-Backoff-Circuit Breaker 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · retry와 breaker의 목적은 다르고 여러 계층이 각자 재시도하면 횟수가 곱셈으로 폭증할 수 있다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · deterministic 4xx까지 재시도하거나 write idempotency 없이 retry하면 부하와 중복 부수효과가 커진다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Timeout-Retry-Backoff-Circuit Breaker을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Timeout-Retry-Backoff-Circuit Breaker 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · A→B→C 각 계층이 3회 retry하여 C 장애 시 요청이 폭증한다.을 재현한 뒤 retry ownership을 한 경계로 제한하고 backoff+jitter, breaker, deadline을 적용한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[AWS Builders' Library](#) · official-primary

[Google SRE Book](#) · official-primary

D24 · Kafka Architecture

Kafka Architecture 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 broker의 정의와 내부 동작을 설명합니다.
- Kafka Architecture에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: broker, topic, partition.
- Kafka Architecture의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · broker · topic · partition · replica · leader · producer · consumer group · offset · rebalance · lag

Learn · 정의에서 관찰까지

정의	Kafka partition은 ordered log이고 ordering 범위는 topic 전체가 아니라 partition 내부다.
왜 필요한가	Kafka Architecture 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Kafka partition은 ordered log이고 ordering 범위는 topic 전체가 아니라 partition 내부다. Consumer group은 partition을 group 내 consumer에 할당하며 offset은 읽기 위치를 나타낸다.
선택 기준	consumer만 늘려도 partition 수보다 많으면 idle이 생긴다. key ordering은 hot partition trade-off가 있 다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- broker와 topic의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- consumer만 늘려도 partition 수보다 많으면 idle이 생긴다. key ordering은 hot partition trade-off가 있 다.

실패 조건

- offset commit이 business 처리 완료보다 먼저 되면 장애 시 처리 누락처럼 보일 수 있다.

관찰

- partition-offset-consumer lag
- producer acknowledgement와 retry 설정
- 중복 처리 방지 key

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	같은 주문의 CREATED/CANCELLED가 key 없이 다른 partition으로 가 순서가 뒤섞인다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. partition-offset-consumer lag producer acknowledgement와 retry 설정 중복 처리 방지 key
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: offset commit이 business 처리 완료보다 먼저 되면 장애 시 처리 누락처럼 보일 수 있다.
조치	orderId key를 사용해 같은 entity ordering을 유지하고 partition skew를 확인한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

같은 주문의 CREATED/CANCELLED가 key 없이 다른 partition으로 가 순서가 뒤섞인다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한단.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 3-partition topic
- key/no-key 분배

독립 단계

- consumer 1/2/4 assignment
- 강제 rebalance
- lag 발생

명령

```
kafka-topics.sh --create --topic study-events --partitions 3 --bootstrap-server localhost:9092
```

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group study-group
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- offset commit이 business 처리 완료보다 먼저 되면 장애 시 처리 누락처럼 보일 수 있다.

관찰 증거

- partition-offset-consumer lag
- producer acknowledgement와 retry 설정
- 중복 처리 방지 key

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. broker의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Kafka partition은 ordered log이고 ordering 범위는 topic 전체가 아니라 partition 내부다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. Kafka Architecture 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · consumer만 늘려도 partition 수보다 많으면 idle이 생긴다. key ordering은 hot partition trade-off가 있다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · offset commit이 business 처리 완료보다 먼저 되면 장애 시 처리 누락처럼 보일 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric-plan-state 중 해당 기술의 증거를 확인합니다.

5. Kafka Architecture을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하 세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Kafka Architecture 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 같은 주문의 CREATED/CANCELLED가 key 없이 다른 partition으로 가 순서가 뒤섞인다.을 재현한 뒤 orderId key를 사용해 같은 entity ordering을 유지하고 partition skew를 확인한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Apache Kafka Documentation](#) · official-primary

[Apache Kafka Documentation](#) · official-primary

D25 · Kafka Reliability·Idempotence·Outbox

Kafka Reliability·Idempotence·Outbox 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 acks의 정의와 내부 동작을 설명합니다.
- Kafka Reliability·Idempotence·Outbox에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: acks, retries, producer idempotence.
- Kafka Reliability·Idempotence·Outbox의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · acks · retries · producer idempotence · at-least-once · idempotent consumer · Outbox · dual-write · CDC

Learn · 정의에서 관찰까지

정의	Producer idempotence는 broker publish retry 중복을 줄이는 범위이고 business idempotency와 다르다.
왜 필요한가	Kafka Reliability·Idempotence·Outbox 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	Producer idempotence는 broker publish retry 중복을 줄이는 범위이고 business idempotency와 다르다. Outbox는 business data와 발행 의도를 같은 DB transaction에 기록해 dual-write gap을 줄인다.
선택 기준	Outbox publisher가 전송 후 status update 전에 죽으면 중복 publish가 가능하므로 consumer idempotency가 여전히 필요하다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- acks와 retries의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- Outbox publisher가 전송 후 status update 전에 죽으면 중복 publish가 가능하므로 consumer idempotency가 여전히 필요하다.

실패 조건

- DB commit 후 Kafka publish 전 crash는 후속 서비스에 event가 전달되지 않는 gap을 만든다.

관찰

- partition-offset-consumer lag
- producer acknowledgement와 retry 설정
- 중복 처리 방지 key

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	주문 DB commit 직후 process가 죽어 Kafka event가 없다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. partition-offset-consumer lag producer acknowledgement와 retry 설정 중복 처리 방지 key
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: DB commit 후 Kafka publish 전 crash는 후속 서비스에 event가 전달되지 않는 gap을 만든다.
조치	order와 outbox를 같은 transaction에 저장하고 publisher/consumer를 idempotent하게 만든다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

주문 DB commit 직후 process가 죽어 Kafka event가 없다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- order+outbox table
- dual-write crash

독립 단계

- publisher 상태
- publish 후 crash 중복
- processed_event unique

명령

```
kafka-console-consumer.sh ... --property print.key=true
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- DB commit 후 Kafka publish 전 crash는 후속 서비스에 event가 전달되지 않는 gap을 만든다.

관찰 증거

- partition-offset-consumer lag
- producer acknowledgement와 retry 설정
- 중복 처리 방지 key

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. acks의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · Producer idempotence는 broker publish retry 중복을 줄이는 범위이고 business idempotency와 다르다. 정의의 뒤 에 실제 목적과 경계를 붙여 설명합니다.

2. Kafka Reliability-Idempotence-Outbox 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · Outbox publisher가 전송 후 status update 전에 죽으면 중복 publish가 가능하므로 consumer idempotency가 여 전히 필요하다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · DB commit 후 Kafka publish 전 crash는 후속 서비스에 event가 전달되지 않는 gap을 만든다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. Kafka Reliability·Idempotence·Outbox을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Kafka Reliability·Idempotence·Outbox 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 주문 DB commit 직후 process가 죽어 Kafka event가 없다.을 재현한 뒤 order와 outbox를 같은 transaction에 저장하고 publisher/consumer를 idempotent하게 만든다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Apache Kafka Documentation](#) · official-primary

[Official or Direct Reference](#) · authoritative-secondary

CHAPTER 17 · 결제·정산·대사·원장

D26 · 결제·정산·대사·원장

결제·정산·대사·원장 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 authorization의 정의와 내부 동작을 설명합니다.
- 결제·정산·대사·원장에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: authorization, cancel, refund.
- 결제·정산·대사·원장의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · authorization · cancel · refund · settlement · reconciliation · ledger · double-entry · idempotency key · cutoff

Learn · 정의에서 관찰까지

정의	금액 처리 시스템은 승인/취소/환불/정산 상태를 분리하고 기준일/cutoff와 adjustment를 명시한다.
왜 필요한가	결제·정산·대사·원장 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	금액 처리 시스템은 승인/취소/환불/정산 상태를 분리하고 기준일/cutoff와 adjustment를 명시한다. 원장은 append-oriented 기록 으로 금액 변화 근거를 남기고 대사는 서로 다른 경계의 누락/중복/금액 차이를 찾는다.
선택 기준	실제 사업자별 용어가 다르므로 범용 상태 machine과 금액 invariant를 먼저 정의한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- authorization와 cancel의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 실제 사업자별 용어가 다르므로 범용 상태 machine과 금액 invariant를 먼저 정의한다.

실패 조건

- 승인 요청 timeout을 곧바로 실패로 처리하면 실제 remote 성공 후 재요청으로 이중 승인이 생길 수 있다.

관찰

- JobInstance-Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.

- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

D26 WORKED EXAMPLE

문제에서 검증까지

문제	승인 timeout 후 client가 같은 결제를 다시 요청한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. JobInstance-Execution 상태 처리 건수와 기준일 부분 실패-재처리 기록
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 승인 요청 timeout을 곧바로 실패로 처리하면 실제 remote 성공 후 재요청으로 이중 승인이 생길 수 있다.
조치	idempotency key와 remote transaction/status query로 결과 미확정 상태를 해결한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

승인 timeout 후 client가 같은 결제를 다시 요청한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB-파일-배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- payment state machine
- 중복 idempotency

독립 단계

- double-entry invariant
- settlement batch
- 누락/중복 대사 report

명령

```
-- GROUP BY business_date, status
```

```
-- ledger debit/credit balance check
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 승인 요청 timeout을 곧바로 실패로 처리하면 실제 remote 성공 후 재요청으로 이중 승인이 생길 수 있다.

관찰 증거

- JobInstance-Execution 상태
- 처리 건수와 기준일
- 부분 실패·재처리 기록

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. authorization의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 금액 처리 시스템은 승인/취소/환불/정산 상태를 분리하고 기준일/cutoff와 adjustment를 명시한다. 정의 뒤에 실제 목적 과 경계를 붙여 설명합니다.

2. 결제·정산·대사·원장에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 실제 사업자별 용어가 다르므로 범용 상태 machine과 금액 invariant를 먼저 정의한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 승인 요청 timeout을 곧바로 실패로 처리하면 실제 remote 성공 후 재요청으로 이중 승인이 생길 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. 결제·정산·대사·원장을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명 하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 결제·정산·대사·원장 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 승인 timeout 후 client가 같은 결제를 다시 요청한다.을 재현한 뒤 idempotency key와 remote transaction/status query로 결과 미확정 상태를 해결한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

Official or Direct Reference · authoritative-secondary

D27 · System Design

System Design 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 requirements의 정의와 내부 동작을 설명합니다.
- System Design에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: requirements, SLO, QPS.
- System Design의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · requirements · SLO · QPS · capacity · API · DB · cache · queue · sharding · observability · failure mode

Learn · 정의에서 관찰까지

정의	설계는 요구사항과 규모를 먼저 확인하고 API→data model→consistency→cache/queue→failure→observability→security 순 으로 구체화한다.
왜 필요한가	System Design 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	설계는 요구사항과 규모를 먼저 확인하고 API→data model→consistency→cache/queue→failure→observability→security 순 으로 구체화한다. Capacity estimate는 정확한 숫자보다 order of magnitude로 병목을 찾는 도구다.
선택 기준	cache/queue/microservice를 기본값처럼 놓지 않고 latency, consistency, traffic, 운영비용이 실제 필요 성을 만든 경우에만 추가한다. 요구 적합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- requirements와 SLO의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- cache/queue/microservice를 기본값처럼 놓지 않고 latency, consistency, traffic, 운영비용이 실제 필요 성을 만든 경우에만 추가한다.

실패 조건

- 정상 경로만 설계하면 DB down, cache miss storm, queue lag 같은 운영 실패를 다루지 못한다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	대량 주문 조회 서비스를 설계한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 정상 경로만 설계하면 DB down, cache miss storm, queue lag 같은 운영 실패를 다루지 못한다.
조치	QPS/최신성/filter/sort/데이터량을 먼저 정하고 RDB+cursor pagination을 시작점으로 삼는다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

대량 주문 조회 서비스를 설계한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 45분 설계
- 10분 요구/용량

독립 단계

- API/data/index
- failure 3개
- trade-off/미측정 가정

명령

-- whiteboard 또는 Mermaid/텍스트 diagram

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 정상 경로만 설계하면 DB down, cache miss storm, queue lag 같은 운영 실패를 다루지 못한다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. requirements의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 설계는 요구사항과 규모를 먼저 확인하고 API→data

model→consistency→cache/queue→failure→observability→security 순으로 구체화한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. System Design 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · cache/queue/microservice를 기본값처럼 놓지 않고 latency, consistency, traffic, 운영비용이 실제 필요성을 만든 경 우에만 추가한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 정상 경로만 설계하면 DB down, cache miss storm, queue lag 같은 운영 실패를 다루지 못한다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. System Design을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. System Design 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 대량 주문 조회 서비스를 설계한다.을 재현한 뒤 QPS/최신성/filter/sort/데이터량을 먼저 정하고 RDB+cursor pagination을 시작점으로 삼는다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Google SRE Book](#) · official-primary

[Google SRE Book](#) · official-primary

D28 · JPA·Redis·Docker·Kubernetes·Security

JPA·Redis·Docker·Kubernetes·Security 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 persistence context의 정의와 내부 동작을 설명합니다.
- JPA·Redis·Docker·Kubernetes·Security에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: persistence context, dirty checking, lazy.
- JPA·Redis·Docker·Kubernetes·Security의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · persistence context · dirty checking · lazy · N+1 · Redis TTL · stampede · container image · Pod · readiness/liveness · secret · OWASP

Learn · 정의에서 관찰까지

정의	JPA는 persistence context에서 entity identity와 dirty checking을 관리하고 lazy association은 N+1을 만들 수 있다.
왜 필요한가	JPA·Redis·Docker·Kubernetes·Security 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	JPA는 persistence context에서 entity identity와 dirty checking을 관리하고 lazy association은 N+1을 만들 수 있다. Redis는 TTL/invalidation/stampede를 운영해야 하며 container/Kubernetes는 image→Pod/Deployment/Service와 readiness/liveness를 구분한다.
선택 기준	모든 association EAGER, TTL 없는 cache, image에 secret 포함, readiness/liveness 혼동은 운영 복잡도 를 키운다. 요구 적합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- persistence context와 dirty checking의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 모든 association EAGER, TTL 없는 cache, image에 secret 포함, readiness/liveness 혼동은 운영 복잡도 를 키운다.

실패 조건

- 연관관계 로딩 정책과 캐시·probe·secret 경계를 구분하지 않으면 쿼리 폭증, stale data, 잘못된 재시작, 비 밀값 노출이 함께 발생할 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	목록 100건을 직렬화하면서 lazy association 100번이 추가 조회된다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 연관관계 로딩 정책과 캐시·probe·secret 경계를 구분하지 않으면 쿼리 폭증, stale data, 잘못된 재시작, 비 밀값 노출이 함께 발생할 수 있다.
조치	필요 필드를 확인해 fetch join/DTO projection 등으로 query 수를 줄이고 result cardinality를 검증한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

목록 100건을 직렬화하면서 lazy association 100번이 추가 조회된다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- JPA N+1
- fetch/DTO 비교

독립 단계

- Redis TTL/miss
- Docker build/run
- K8s probe 읽기
- secret scan

명령

```
docker build -t backend-study .
```

```
docker run --rm -p 8080:8080 backend-study
```

```
redis-cli TTL sample:key
```

```
kubectrl explain deployment.spec.template.spec.containers.readinessProbe
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 연관관계 로딩 정책과 캐시·probe-secret 경계를 구분하지 않으면 쿼리 폭증, stale data, 잘못된 재시작, 비 밀값 노출이 함께 발생할 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. persistence context의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · JPA는 persistence context에서 entity identity와 dirty checking을 관리하고 lazy association은 N+1을 만들 수 있다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. JPA·Redis·Docker·Kubernetes·Security 에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 모든 association EAGER, TTL 없는 cache, image에 secret 포함, readiness/liveness 혼동은 운영 복잡도를 키운다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 연관관계 로딩 정책과 캐시·probe-secret 경계를 구분하지 않으면 쿼리 폭증, stale data, 잘못된 재시작, 비밀값 노출이 함께 발생할 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. JPA·Redis·Docker·Kubernetes·Security을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. JPA·Redis·Docker·Kubernetes·Security 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 목록 100건을 직렬화하면서 lazy association 100번이 추가 조회된다.을 재현한 뒤 필요 필드를 확인해 fetch join/DTO projection 등으로 query 수를 줄이고 result cardinality를 검증한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Official or Direct Reference](#) · official-primary

[Redis Documentation](#) · official-primary

[Docker Documentation](#) · official-primary

[Kubernetes Documentation](#) · official-primary

[OWASP Guidance](#) · official-primary

D29 · AI·RAG 를 사용하는 백엔드

AI·RAG 를 사용하는 백엔드 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 LLM의 정의와 내부 동작을 설명합니다.
- AI·RAG를 사용하는 백엔드에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: LLM, embedding, vector.
- AI·RAG 를 사용하는 백엔드의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · LLM · embedding · vector · chunk · retrieval · RAG · grounding · provider timeout · rate limit · agent · approval

Learn · 정의에서 관찰까지

정의	RAG는 문서 chunk/index→query retrieval→context→model generation→source/validation 의 pipeline이다.
왜 필요한가	AI·RAG 를 사용하는 백엔드 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	RAG는 문서 chunk/index→query retrieval→context→model generation→source/validation 의 pipeline이다. Vector search는 similarity retrieval이지 사실 보장이 아니며 provider API는 timeout/429/quota/model 변경을 가진 외부 dependency다.
선택 기준	AI 기능을 core transaction과 분리해 실패 시 부분 DEGRADED로 만들고 mutation tool에는 approval/audit/allowlist를 둔다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- LLM와 embedding의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- AI 기능을 core transaction과 분리해 실패 시 부분 DEGRADED로 만들고 mutation tool에는 approval/audit/allowlist를 둔다.

실패 조건

- provider 장애가 core admin request path에 강결합되면 AI 기능 하나 때문에 전체 서비스가 500이 될 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	운영 문서 질의 provider 장애 때문에 관리 화면 전체가 실패한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: provider 장애가 core admin request path에 강결합되면 AI 기능 하나 때문에 전체 서비스가 500이 될 수 있다.
조치	RAG만 DEGRADED로 격리하고 근거가 없는 답변은 "근거 없음"으로 처리한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

운영 문서 질의 provider 장애 때문에 관리 화면 전체가 실패한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- Markdown chunking
- toy vector retrieval

독립 단계

- source 포함 API
- provider timeout/429
- read-only/mutation tool 분리

명령

```
curl -X POST http://localhost:8080/rag/search -H "Content-Type: application/json" -d '{"q":"transaction"}'
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- provider 장애가 core admin request path에 강결합되면 AI 기능 하나 때문에 전체 서비스가 500이 될 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. LLM의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · RAG는 문서 chunk/index→query retrieval→context→model generation→source/validation 의 pipeline이다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. AI-RAG 를 사용하는 백엔드에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · AI 기능을 core transaction과 분리해 실패 시 부분 DEGRADED로 만들고 mutation tool에는 approval/audit/allowlist를 둔다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · provider 장애가 core admin request path에 강결합되면 AI 기능 하나 때문에 전체 서비스가 500이 될 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그·metric·plan·state 중 해당 기술의 증거를 확인합니다.

5. AI-RAG를 사용하는 백엔드을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. AI-RAG 를 사용하는 백엔드 주제를 트래픽 증가와 부분 장애 조건에서도 운영 가능하도록 설계하고 불변식·관측·rollback을 설명하세요.

유형 system-design · 난이도 hard

기준 답안 · 운영 문서 질의 provider 장애 때문에 관리 화면 전체가 실패한다.을 재현한 뒤 RAG만 DEGRADED로 격리하고 근거가 없는 답변은 “근거 없음”으로 처리한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Spring AI Reference](#) · official-primary

[Official or Direct Reference](#) · official-primary

[Official or Direct Reference](#) · official-primary

D30 · 실무 장애·성능 사례 분석

실무 장애·성능 사례 분석 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 symptom의 정의와 내부 동작을 설명합니다.
- 실무 장애·성능 사례 분석에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: symptom, hypothesis, evidence.
- 실무 장애·성능 사례 분석의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · symptom · hypothesis · evidence · root cause · mitigation · permanent fix · verification · rollback · regression

Learn · 정의에서 관찰까지

정의	사례는 증상→영향→가설→증거→root cause→임시 완화→영구 조치→검증→한계 순으로 기록한다.
왜 필요한가	실무 장애·성능 사례 분석 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	사례는 증상→영향→가설→증거→root cause→임시 완화→영구 조치→검증→한계 순으로 기록한다. 성능은 입력 건수가 다르면 throughput 등 정규화 지표를 쓰고 측정하지 않은 숫자는 만들지 않는다.
선택 기준	SQL 한 줄보다 데이터 lifecycle/write pattern 자체를 바꾸는 것이 필요한 문제도 있다. 환경/IDC 전환은 Java/DB/Batch/EAI/File/TLS/Nginx/데이터를 전수 확인한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- symptom와 hypothesis의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- SQL 한 줄보다 데이터 lifecycle/write pattern 자체를 바꾸는 것이 필요한 문제도 있다. 환경/IDC 전환은 Java/DB/Batch/EAI/File/TLS/Nginx/데이터를 전수 확인한다.

실패 조건

- 최근 변경과 기준선을 남기지 않으면 원인과 개선을 증명하기 어렵고 재처리 시 중복을 만들 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.

- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	반복 INSERT/DELETE와 대량 DELETE, thread/DB contention으로 대량 처리 지연이 커진다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 최근 변경과 기준선을 남기지 않으면 원인과 개선을 증명하기 어렵고 재처리 시 중복을 만들 수 있다.
조치	append-oriented 적재와 시간 partition, worker 설정과 plan을 각각 검증하고 정합성 대사를 수행한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

반복 INSERT/DELETE와 대량 DELETE, thread/DB contention으로 대량 처리 지연이 커진다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 장애 3개 postmortem
- throughput 계산

독립 단계

- migration checklist
- snapshot/correlation runbook
- 미측정/향후지표 분리

명령

```
-- 처리량 = 처리건수 / 실행시간
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 최근 변경과 기준선을 남기지 않으면 원인과 개선을 증명하기 어렵고 재처리 시 중복을 만들 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. symptom의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 사례는 증상→영향→가설→증거→root cause→임시 완화→영구 조치→검증→한계 순으로 기록한다. 정의 뒤에 실제 목 적과 경계를 붙여 설명합니다.

2. 실무 장애·성능 사례 분석에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · SQL 한 줄보다 데이터 lifecycle/write pattern 자체를 바꾸는 것이 필요한 문제도 있다. 환경/IDC 전환은 Java/DB/Batch/EAI/File/TLS/Nginx/데이터를 전수 확인한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 최근 변경과 기준선을 남기지 않으면 원인과 개선을 증명하기 어렵고 재처리 시 중복을 만들 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. 실무 장애·성능 사례 분석을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설 명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 반복 INSERT/DELETE와 대량 DELETE, thread/DB contention으로 대량 처리 지연이 커진다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 hard

기준 답안 · 반복 INSERT/DELETE와 대량 DELETE, thread/DB contention으로 대량 처리 지연이 커진다.을 재현한 뒤 append-oriented 적재와 시간 partition, worker 설정과 plan을 각각 검증하고 정합성 대사를 수행한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Google SRE Book](#) · official-primary

[Google SRE Book](#) · official-primary

D31 · 통합 역량 시험

통합 역량 시험 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 integrated assessment의 정의와 내부 동작을 설명합니다.
- 통합 역량 시험에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: integrated assessment, fact vs assumption, constraint.
- 통합 역량 시험의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · integrated assessment · fact vs assumption · constraint · trade-off · failure path · verification

Learn · 정의에서 관찰까지

정의	여러 영역을 하나의 시나리오에 연결한다.
왜 필요한가	통합 역량 시험 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	여러 영역을 하나의 시나리오에 연결한다. 문제를 받으면 사실/가정/확인할 지표를 분리하고 Java는 복잡도·edge, SQL은 정합성·plan, 설계는 failure/metric까지 포함한다.
선택 기준	모르는 부분은 추측으로 채우지 않고 현재 아는 범위와 확인 방법을 제시한다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- integrated assessment와 fact vs assumption의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 모르는 부분은 추측으로 채우지 않고 현재 아는 범위와 확인 방법을 제시한다.

실패 조건

- 한 영역의 정답만 고집하면 실제 원인이 DB lock, thread pool, EAI backlog처럼 다른 경계에 있을 수 있다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.
- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	정산 마감 batch가 느려지고 외부 송신도 지연된다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그·metric·state 정상·경계·실패·재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 한 영역의 정답만 고집하면 실제 원인이 DB lock, thread pool, EAI backlog처럼 다른 경계에 있을 수 있다.
조치	데이터량→Step 시간→plan/lock→thread/pool→EAI backlog→외부 응답 순으로 기준선과 비교한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상·경계·실패·재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

정산 마감 batch가 느려지고 외부 송신도 지연된다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- Java2 45분
- SQL2 35분

독립 단계

- System Design 45분
- 장애 시나리오 30분
- 자기채점/24시간 재시험

명령

-- 총 155분 timebox

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 한 영역의 정답만 고집하면 실제 원인이 DB lock, thread pool, EAI backlog처럼 다른 경계에 있을 수 있다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. integrated assessment의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 여러 영역을 하나의 시나리오에 연결한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. 통합 역량 시험에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 모르는 부분은 추측으로 채우지 않고 현재 아는 범위와 확인 방법을 제시한다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 한 영역의 정답만 고집하면 실제 원인이 DB lock, thread pool, EAI backlog처럼 다른 경계에 있을 수 있다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan-state 중 해당 기술의 증거를 확인합니다.

5. 통합 역량 시험을(를) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. 정산 마감 batch가 느려지고 외부 송신도 지연된다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 hard

기준 답안 · 정산 마감 batch가 느려지고 외부 송신도 지연된다.을 재현한 뒤 데이터량→Step

시간→plan/lock→thread/pool→EAI backlog→외부 응답 순으로 기준선과 비교한다. 조치를 적용하고

정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

Google SRE Book · official-primary

D32 · 누적 회상·약점 복구

누적 회상·약점 복구 주제를 API 이름이 아니라 입력, 처리, 상태 변화, 출력과 실패 경계로 설명하고 재현 가능한 증거로 검증한다.

학습 목표 · 사전 진단

- 핵심 용어인 retrieval의 정의와 내부 동작을 설명합니다.
- 누적 회상·약점 복구에서 선택 기준과 trade-off를 설명합니다.
- 실패 조건을 직접 재현하고 어떤 로그/상태/계획을 확인할지 말합니다.

사전 진단

- 다음 핵심 용어를 보지 않고 정의한다: retrieval, spacing, interleaving.
- 누적 회상·약점 복구의 대표 실패를 어떤 증거로 분리할지 말한다.

핵심 용어 · retrieval · spacing · interleaving · error log · knowledge map · weakness queue

Learn · 정의에서 관찰까지

정의	노트를 다시 읽기 전에 기억에서 핵심 흐름을 꺼내고 빠진 부분만 보충한다.
왜 필요한가	누적 회상·약점 복구 학습은 단순 API 사용법보다 시스템 경계와 상태 변화의 원리를 알아야 장애·성능 문제를 올바르게 진단할 수 있습니다. 코드나 설정을 먼저 바꾸기보다 현재 상태와 기대 invariant를 정의한 뒤 증거를 모으는 습관을 사용합니다.
내부 동작	노트를 다시 읽기 전에 기억에서 핵심 흐름을 꺼내고 빠진 부분만 보충한다. 오답은 날짜보다 원인 tag로 묶고 1/3/7/14일 같은 간격으로 재시험한다.
선택 기준	초기 학습은 worked example과 같은 유형 연습이 도움이 되고 숙련 후에는 interleaving으로 전략 선택 능력 을 키운다. 요구 정합성, 동시성, 지연, 운영 복잡도와 rollback 가능성을 함께 보고 선택한다.

비교

- retrieval와 spacing의 책임·상태·실패 경계를 같은 입력으로 비교한다.

Trade-off

- 초기 학습은 worked example과 같은 유형 연습이 도움이 되고 숙련 후에는 interleaving으로 전략 선택 능력 을 키운다.

실패 조건

- 이미 아는 정의를 계속 읽는 대신 반복 실패하는 연결을 찾지 않으면 약점이 그대로 남는다.

관찰

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

코드·명령 읽기

- 명령이나 코드가 바꾸는 상태와 대상 범위를 먼저 표시한다.
- 정상 결과와 실패 시 첫 관찰 지점을 실행 전에 적는다.

- 측정값은 동일 입력·시간 단위·처리 건수로 정규화한다.

문제에서 검증까지

문제	Transaction 정의는 알지만 self-invocation과 propagation에서 반복 실패한다.
관찰	증상 하나로 원인을 단정하지 않습니다. 최근 변경, 입력 크기, 오류 문맥, 상태/로그/메트릭을 비교해 가설을 줄입니다. 이 예에서는 핵심 제약과 실패 경계를 먼저 표시합니다. 입력과 예상 invariant 변경 전후 로그-metric-state 정상-경계-실패-재실행 결과
가설	최근 변경과 입력 차이를 먼저 배제한다. 다음 실패 조건을 최소 재현으로 확인한다: 이미 아는 정의를 계속 읽는 대신 반복 실패하는 연결을 찾지 않으면 약점이 그대로 남는다.
조치	D10 실습을 다시 실행하고 1일/3일 뒤 관련 질문만 재시험해 취약 연결을 복구한다.
검증	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다. 정상-경계-실패-재실행 결과를 같은 기준으로 보관한다.
대안	대안은 단순히 다른 기술 이름을 나열하지 않고 "어떤 조건이면 더 나은가"를 적습니다. 변경 후에는 정상 입력, 경계 입력, 실패 입력, 재실행을 검증하고 결과 값뿐 아니라 관련 로그/상태/실행계획을 함께 보관합니다.

Guided Practice · Independent Practice

목표

Transaction 정의는 알지만 self-invocation과 propagation에서 반복 실패한다. 상황을 정상 기준선부터 재현하고 조치의 인과관계를 증명한다.

준비

- 실습 폴더를 만들고 현재 버전/환경을 notes.md에 기록합니다.
- 정상 결과를 먼저 저장한 뒤 실패를 의도적으로 만듭니다.
- DB·파일·배포 변경은 원복 절차를 먼저 적습니다.
- 실패 시 첫 오류만 보지 않고 원인 후보를 2~3개로 분리합니다.

안내 단계

- 32개 핵심 1줄 회상
- 오답 Top20 tag

독립 단계

- 약점5 실습 재현
- 누적 30문항
- 4주 일정 작성

명령

```
-- notes/assessment 데이터 정리
```

실패를 만들고, 원인을 설명하고, 회귀를 막으세요

실패 주입

- 이미 아는 정의를 계속 읽는 대신 반복 실패하는 연결을 찾지 않으면 약점이 그대로 남는다.

관찰 증거

- 입력과 예상 invariant
- 변경 전후 로그·metric·state
- 정상·경계·실패·재실행 결과

검증

- 정상 결과와 실패 결과를 각각 보관합니다.
- 변경 전후 동일한 입력 또는 정규화 기준으로 비교합니다.
- 원인과 조치의 인과관계를 말로 설명합니다.
- 재실행/rollback 후 상태가 기대한 invariant로 돌아오는지 확인합니다.

회귀

- 정상 입력을 다시 실행해 결과가 유지되는지 확인한다.
- 실패 주입을 제거한 뒤 상태가 기준선으로 돌아오는지 확인한다.

완료 기준

- 모든 단계의 입력과 결과가 notes.md에 남아 있다.
- 원인과 조치의 인과관계를 설명할 수 있다.
- 대안 하나와 적용하지 않을 조건을 말할 수 있다.

항목	배점
correctness	40
explanation	20
failureReproduction	15
implementationQuality	10
evidence	10
tradeoff	5

Quiz · Explain

1. retrieval의 정의와 필요한 이유는 무엇인가?

유형 recall · 난이도 easy

기준 답안 · 노트를 다시 읽기 전에 기억에서 핵심 흐름을 꺼내고 빠진 부분만 보충한다. 정의 뒤에 실제 목적과 경계를 붙여 설명합니다.

2. 누적 회상·약점 복구에서 대표적인 trade-off 는?

유형 mcq · 난이도 easy

기준 답안 · 초기 학습은 worked example과 같은 유형 연습이 도움이 되고 숙련 후에는 interleaving으로 전략 선택 능력을 키운다.

3. 어떤 실패 조건을 우선 확인해야 하는가?

유형 scenario · 난이도 medium

기준 답안 · 이미 아는 정의를 계속 읽는 대신 반복 실패하는 연결을 찾지 않으면 약점이 그대로 남는다.

4. 위 Worked Example 에서 조치 후 무엇으로 검증해야 하는가?

유형 short-answer · 난이도 medium

기준 답안 · 정상/경계/실패 입력과 로그:metric:plan:state 중 해당 기술의 증거를 확인합니다.

5. 누적 회상·약점 복구율(률) 처음 듣는 동료에게 정의 → 필요한 이유 → 내부 동작 → 실패 조건 → 검증 방법 순서로 설명하 세요.

유형 oral · 난이도 medium

기준 답안 · 정의, 필요한 이유, 내부 동작, 실패 조건, 검증 방법 순서로 60~90초 안에 설명한다.

6. Transaction 정의는 알지만 self-invocation과 propagation에서 반복 실패한다. 증상을 변경 없이 먼저 진단하는 증거 수집 순서를 작성하세요.

유형 debug · 난이도 hard

기준 답안 · Transaction 정의는 알지만 self-invocation과 propagation에서 반복 실패한다.을 재현한 뒤 D10 실습을 다시 실행하고 1일/3일 뒤 관련 질문만 재시험해 취약 연결을 복구한다. 조치를 적용하고 정상·경계·실패·재실행 증거를 비교해야 한다.

공식 자료

[Retrieval and Spacing Research](#) · peer-reviewed-direct

[Retrieval and Spacing Research](#) · peer-reviewed-direct

완주 후 종합 점검

32일을 모두 끝낸 뒤에는 정의를 외우는 데서 멈추지 말고, 같은 상황을 재현하고 로그·메트릭·상태·실행계획으로 원인을 분리한 뒤 대안과 적용하지 않을 조건까지 설명하세요.

회상 체크

- 정의 없이 사례만 말하지 않았는가
- 내부 상태 변화와 실패 경계를 설명했는가
- 대안의 선택 조건과 비용을 비교했는가
- 정상·경계·실패·재실행 증거를 남겼는가
- 1·3·7·14일 간격으로 다시 설명했는가